



Semantic sensitive tensor factorization

Makoto Nakatsuji^{a,*}, Hiroyuki Toda^b, Hiroshi Sawada^b, Jin Guang Zheng^c,
James A. Hendler^c

^a Smart Navigation Business Division, NTT Resonant Inc., Japan

^b NTT Service Evolution Laboratories, NTT Corporation, Japan

^c Tetherless World Constellation, Rensselaer Polytechnic Institute, USA

ARTICLE INFO

Article history:

Received 18 August 2014

Received in revised form 28 August 2015

Accepted 1 September 2015

Available online 14 September 2015

Keywords:

Tensor factorization

Linked open data

Recommendation systems

ABSTRACT

The ability to predict the activities of users is an important one for recommender systems and analyses of social media. User activities can be represented in terms of relationships involving three or more things (e.g. when a user tags items on a webpage or tweets about a location he or she visited). Such relationships can be represented as a tensor, and tensor factorization is becoming an increasingly important means for predicting users' possible activities. However, the prediction accuracy of factorization is poor for ambiguous and/or sparsely observed objects. Our solution, Semantic Sensitive Tensor Factorization (SSTF), incorporates the semantics expressed by an object vocabulary or taxonomy into the tensor factorization. SSTF first links objects to classes in the vocabulary (taxonomy) and resolves the ambiguities of objects that may have several meanings. Next, it lifts sparsely observed objects to their classes to create augmented tensors. Then, it factorizes the original tensor and augmented tensors simultaneously. Since it shares semantic knowledge during the factorization, it can resolve the sparsity problem. Furthermore, as a result of the natural use of semantic information in tensor factorization, SSTF can combine heterogeneous and unbalanced datasets from different Linked Open Data sources. We implemented SSTF in the Bayesian probabilistic tensor factorization framework. Experiments on publicly available large-scale datasets using vocabularies from linked open data and a taxonomy from WordNet show that SSTF has up to 12% higher accuracy in comparison with state-of-the-art tensor factorization methods.

© 2015 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The ability to analyze relationships involving three or more objects is critical for accurately predicting human activities. An example of a typical relationship involving three or more objects in a content providing service is one between a user, an item on a webpage, and a user-assigned tag of that item. Another example is the relationship between a user, his or her tweet on Twitter, and the locations at which he or she tweeted. The ability to predict such relationships can be used to improve recommendation systems and social network analysis. For example, suppose that a user assigns a thriller movie the tag “romance” and another user tags it with “car action”. Here, methods that only handle bi-relational objects ignore tags and find these users to be similar because they mentioned the same item. In contrast, multi-relational methods conclude

* Corresponding author.

E-mail address: nakatsuji.makoto@gmail.com (M. Nakatsuji).

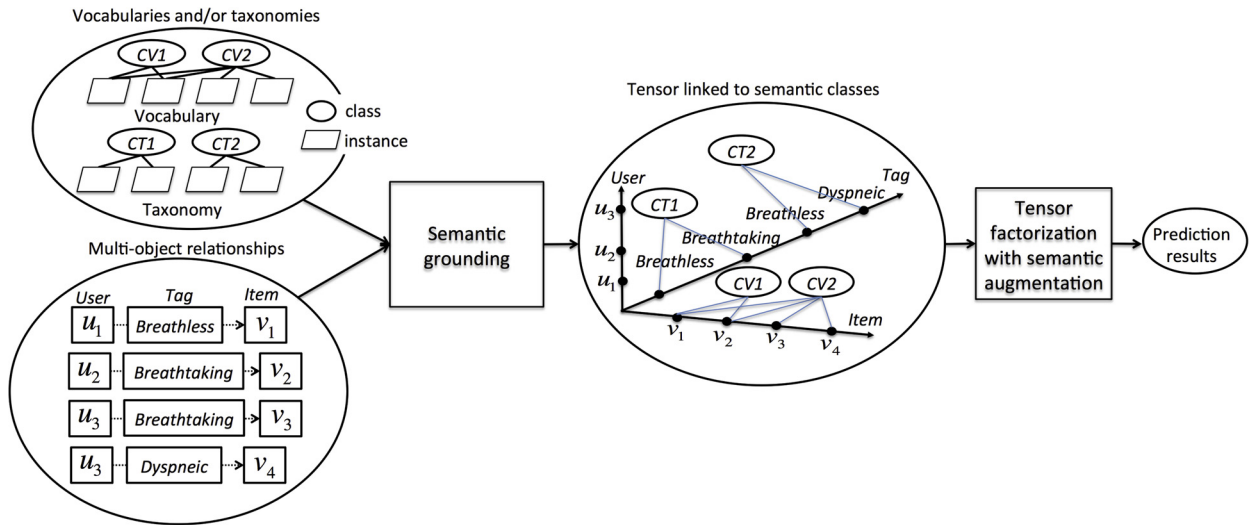


Fig. 1. Flowchart of SSTF.

that such users are slightly different because they have different opinions about the item. The quality of recommendations would be higher if they reflected these small differences [1].

Tensors are useful ways of representing relationships involving three or more objects, and tensor factorization is seen as a means of predicting possible future relationships [2]. Bayesian probabilistic tensor factorization (BPTF) [3] is especially promising because of its efficient sampling of large-scale datasets and simple parameter settings. However, this and other tensor factorization schemes have had poor accuracy because they fail to utilize the semantics underlying the objects and have trouble handling ambiguous and/or sparsely observed objects.

Semantic ambiguity is a fundamental problem in text clustering. Several studies have used Wikipedia or WordNet taxonomies [4] to resolve semantic ambiguities and improve the performance of text clustering [5] and to compute the semantic relatedness of documents [6]. We show in this paper that taxonomy-based disambiguation can improve the prediction accuracy of tensor factorization.

The *sparsity problem* affects accuracy if the dataset used for learning latent features in tensor factorizations is not sufficient [7]. In an attempt to improve prediction accuracy, generalized coupled tensor factorization (GCTF) [8,9] uses social relationships among users as auxiliary information in addition to user–item–tag relationships during the tensor factorization. Recently, GCTF was used for link prediction [10], and it was found to be the most accurate of the current tensor factorization methods that use auxiliary information [7,11,12]. However, so far, no tensor methods have used semantics expressed as taxonomies to resolve ambiguity/sparsity problems even though taxonomies are present in real applications as a result of the spread of the Linked Open Data (LOD) vision [13] and the growing knowledge graphs used in search.¹

In this paper, we propose semantic sensitive tensor factorization (SSTF), which uses semantics expressed by vocabularies and taxonomies to overcome the above problems in the BPTF framework. Vocabularies and taxonomies, sometimes called “simple ontologies” [14], are collections of human-defined classes with a hierarchical structure and classification instances (i.e., items or words). We will disambiguate the objects (items or tags) by using “vocabularies” for graph structures and “taxonomies” for tree structures. Fig. 1 overviews SSTF. The factorization has two components, *semantic grounding* and *tensor factorization with semantic augmentation*, which respectively resolve the semantic ambiguity and sparsity problems.

Semantic grounding resolves semantic ambiguities by linking objects to vocabularies or taxonomies. It first measures the similarity between objects and instances in the vocabularies (taxonomies). It then links each object to vocabulary (taxonomy) classes via the instance that is most similar to the object. Consequently, it can construct a tensor whose objects are linked to classes. For example, in Fig. 1, the tag “Breathless” can be linked to the class CT1, which includes word instances such as “Breathtaking” and “Breathless” or CT2, which includes the word instances such as “Dyspneic” and “Breathless”. CT1 and CT2 have different meanings, and a tensor factorization trained on observed relationships that include such ambiguous tags can degrade prediction accuracy. SSTF extracts the properties that are assigned to item entry v_1 in LOD sources and those assigned to instances in WordNet. Then, it links the tag “Breathless” to CT1 if the properties of “Breathless” in CT1 are more similar to the properties of v_1 than to those of “Breathless” in CT2. We describe this process in detail in Section 4.2.

Tensor factorization with semantic augmentation solves the sparsity problem by incorporating semantic biases based on vocabulary (taxonomy) classes into tensor factorization. The key point of this idea is that it lets sparse objects share semantic knowledge with regard to their classes during the factorization process. It lifts sparse objects to their classes to create augmented tensors. To do this, it determines multiple sets of sparse objects according to the degree of sparsity to create

¹ <http://www.google.com/insidesearch/features/search/knowledge.html>.

multiple augmented tensors. It then factorizes the original tensor and the augmented tensors simultaneously to compute feature vectors for objects and for classes. By factorizing multiple augmented tensors, it creates feature vectors for classes according to the degree of sparsity. As a result, through these feature vectors, relationships between sparse objects can include detailed semantic knowledge with regard to their classes. This aims to solve the sparsity problem. For example, in Fig. 1, items v_1 and v_2 are linked to classes CV1 and CV2, and items v_3 and v_4 are linked to class CV2. Suppose that items v_1 and v_2 are in a set that includes the most sparse items. Item v_3 is in a set that includes the second-most sparse items. In this case, v_1 and v_2 can share the semantic knowledge in CV1 and CV2 with each other. Moreover, v_3 can share the semantic knowledge with v_1 and v_2 in CV2. This semantic knowledge ameliorates the sparsity problem in tensor factorization. Furthermore, SSTF can combine heterogeneous and unbalanced datasets though it factorizes the original tensor and augmented tensors, which are created from different data sources, simultaneously. This is because, in creating the augmented tensors, SSTF inserts relationships composed of semantic classes of sparsely observed objects (i.e. rating relationships composed of users, item classes, and tags) into the original tensor. Those augmented relationships are of the same type present in the original tensor. This semantic augmentation is described in detail in Section 4.3.

Thus, our approach leverages semantics to incorporate expert knowledge into a sophisticated machine learning scheme, tensor factorization. SSTF resolves object ambiguities by linking objects to vocabulary and taxonomy classes. It also incorporates semantic biases into tensor factorization by sensitively tracking the degree of sparsity of objects while factorizing original tensor and augmented tensors simultaneously; the sparsity problem is solved by sharing the semantic knowledge among sparse objects. As a result, SSTF achieves much higher accuracy than the current best methods. We implemented SSTF in the BPTF framework. Thus, it inherits the benefits of a Bayesian treatment of tensor factorization, i.e., easy parameter settings and fast computation by parallel computation of feature vectors.

We evaluated our method on three datasets: (1) MovieLens ratings/tags² with FreeBase³/WordNet, (2) Yelp ratings/reviews⁴ with DBpedia [15]/WordNet, and (3) Last.fm listening counts/tags⁵ with FreeBase and WordNet. We put the datasets and Matlab code of our proposal online, so readers can use them to easily confirm our results and proceed with their own studies.⁶ The results show that SSTF has 6–12% higher accuracy in comparison with state-of-the-art methods including GCTF. It suits many applications (i.e. movie rating/tagging systems, restaurant rating/review systems, and music listening/tagging systems) that manage various items since the growing knowledge graph means that applications increasingly have semantic knowledge underlying objects.

The paper is organized as follows: Section 2 describes related work, and Section 3 introduces the background of this paper. Section 4 explains our method, and Section 5 evaluates it. Section 6 concludes the paper.

2. Related work

Tensor factorization methods are used in various applications such as recommendation systems [2,16], semantic web analytics [17,18], and social network analysis [11,19,20]. For example, [2] models observation data as a user–item–context tensor to provide context-aware recommendations. TripleRank [17] analyzes Linked Open Data (LOD) triples by using tensor factorization and ranks authoritative sources with semantically coherent predicates. [20,21] provide personal tag recommendations by factorizing tensors composed of users, items, and social tags.

This section first details the tensor factorization studies that utilize the side information in their factorization process to solve the sparsity problem. It next describes efficient tensor factorization schemes and then introduces tensor studies that use semantic data including linked open data to analyze or enhance factorized results. Other than the tensor-based methods, it finally explains methods that use a tripartite graph to compute the link or rating predictions.

2.1. Coupled tensor factorization

One major problem with tensor factorization is that its prediction accuracy tends to be poor because observations made in real datasets are typically sparse [7]. Several matrix factorization studies have tried to incorporate side information such as social connections among users to improve factorization accuracy [22]. However, their focus is on predicting paired-object relationships. As a result, they differ in purpose from ours, i.e., predicting possible future relationships between three or more objects.

Coupled tensor factorization algorithms [7,8,11,12,16,23–26] have recently gained attention. They represent relationships composed of three or more objects (e.g. rating relationships involving users, items, and tags) as a tensor and side information (e.g. link relationships composed by items and their properties) as matrices or tensors. They associate objects (e.g. items) with tensors and matrices, each of which treats different types of information (e.g. tensors represent rating relationships whereas matrices represent link relationship); this often triggers a balance problem when handling heterogeneous

² Available at <http://www.grouplens.org/node/73>.

³ <http://www.freebase.com>.

⁴ Available at http://www.yelp.com/dataset_challenge/.

⁵ Available at <http://www.grouplens.org/node/462>.

⁶ Available at <https://sites.google.com/site/sbjtax/sstf>.

datasets [27]. Then, they simultaneously factorize the tensors and matrices to make predictions. Among them, generalized coupled tensor factorization (GCTF) [8] utilizes the well-established theory of generalized linear models to factorize multiple observed tensors simultaneously. It can compute arbitrary factorizations in a message passing framework derived for a broad class of exponential family distributions. GCTF has been applied to social network analysis [9], musical source separation [28], and image annotation [29] and it has been proven useful in joint analysis of data from multiple sources. Acar et al. have formulated a coupled matrix and tensor factorization (CMTF) problem, where heterogeneous datasets are modeled by fitting outer-product models to higher-order tensors and matrices in a coupled manner. They proposed an all-at-once optimization approach called CMTF-OPT (CMTF-OPTimization), which is a gradient-based optimization approach for joint analysis of matrices and higher-order tensors [12]. They used their method to analyze metabolomics datasets and found that they can capture the underlying sparse patterns in data [30]. Furthermore, on a real data set of blood samples collected from a group of rats, They used CMTF-OPT to jointly analyze metabolomics datasets and identify potential biomarkers for apple intake. They also reported the limitations of coupled tensor factorization analysis, not only its advantages [31]. For instance, coupled analysis performs worse than the analysis of a single dataset when the underlying common factors constitute only a small part of the data under inspection or when the shared factors only contribute a little to each dataset. To solve the sparsity problem affecting tensor completion, [23] proposed non-negative multiple tensor factorization (NMTF), which factorizes the target tensor and auxiliary tensors simultaneously, where auxiliary data tensors compensate for the sparseness of the target data tensor. More recently, [32] proposed a method that uses common structures in datasets to improve the prediction performance. Instead of common latent factors in factorization, it assumes that datasets share a common adjacency graph (CAG) structure, which is more able to deal with heterogeneous and unbalanced datasets.

SSTF differs from the above studies mainly in the following four senses: (1) it presents the way of incorporating semantic information, which is now available in the format of linked open data, into tensor factorization. The evaluations show that our ideas, semantic grounding and tensor factorization with semantic augmentation, are very useful in improving prediction results in tensor factorization. (2) From the viewpoint of coupled tensor factorization, SSTF is more robust to deal with heterogeneous and unbalanced datasets, as demonstrated in the evaluation section. This is because it augments a tensor by inserting relationships composed of semantic classes of sparsely observed objects (i.e. rating relationships composed of users, item classes, and tags) into the original tensor. These augmented relationships are of the same type present in the original tensor, so SSTF can naturally combine and simultaneously factorize the original tensor and augmented tensors. Thus, its approach is suitable for combining heterogeneous datasets in factorization such as ratings datasets and link relationships datasets. (3) It focuses on multi-object relationships including sparse objects. Thus, it can effectively solve the sparsity problem while avoiding the useless semantic information from objects that are observed a lot. (4) It exploits the Bayesian approach in factorizing multiple tensors simultaneously. Thus, it can avoid over-fitting in the optimization with easy parameter settings. The Bayesian approaches have reported to be more accurate than non-Bayesian ones in rating prediction applications [3].

2.2. Efficient tensor factorization

Recently, an efficient tensor factorization method based on a probabilistic framework has emerged [3,33]. By introducing priors on the parameters, BPTF [3] can effectively average over various models and lower the cost of tuning the parameters (see Section 3). As for scalability, it offers an efficient Markov chain Monte Carlo (MCMC) procedure for the learning process, so it can be applied to large-scale datasets like Netflix.⁷

Several studies have tried to compute predictions efficiently by using coupled tensor factorization. [34] presented a distributed, scalable method for decomposing matrices, tensors, and coupled data sets through stochastic gradient descent on a variety of objective functions. [35] introduced a parallel and distributed algorithmic framework for coupled tensor factorization to simultaneously estimate latent factors, specific divergences for each dataset, as well as the relative weights in an overall additive cost function. [36] proposed Turbo-SMT, which boosts the performance of coupled tensor factorization algorithms while maintaining their accuracy.

2.3. Tensor factorization using semantic data

As ways of handling linked open data, [18,37] proposed methods that analyze a huge volume of LOD datasets by tensor factorization in a reasonable amount of time. They, however, did not use a coupled tensor factorization approach and thus could not explicitly incorporate the semantic relationships behind multi-object relationships into the tensor factorization; in particular, they could not use taxonomical relationships behind multi-object relationships such as “subClassOf” and “subGenreOf”, which are often seen in LOD datasets.

Taxonomies have been used to solve the sparsity problem affecting memory-based collaborative filtering [1,38–42]. Model-based methods such as matrix and tensor factorization, however, usually achieve higher accuracy than memory-based methods do [43]. Taxonomies are also used in genome science for clustering genomes with functional annotations [44].

⁷ <https://signup.netflix.com/global>.

Thus, their use is promising for analyzing such datasets as well as user activity datasets. There are, however, no tensor factorization studies that use vocabularies/taxonomies to improve prediction accuracy.

We previously proposed Semantic data Representation for Tensor Factorization (SRTF) [16] that incorporates semantic biases into tensor factorization to solve the sparsity problem. SRTF is the first method that incorporates semantics behind multi-object relationships into tensor factorization. Furthermore, SRTF can be considered the first coupled tensor factorization as that uses the Bayesian approach to simultaneously factorize several tensors for rating predictions. Khan and Kaski [45] presented a Bayesian extension of the tensor factorization of multiple coupled tensors after our publication [16]. This paper extends SRTF by introducing the degrees of sparsity of objects and adjusting how much the semantic biases should be incorporated into tensor factorization by sensitively tracking those degrees. This paper gives a much more detailed explanation of the Markov chain Monte Carlo (MCMC) procedure for SSTF than the one for SRTF in [16]. It also describes detailed evaluations that used a dataset on user listening frequencies as well as datasets on user rating activities. These evaluations show that SSTF is much more accurate than SRTF because it sensitively gives semantic biases to sparsely observed objects in tensor factorization according to the degree of sparsity of the objects.

2.4. Methods that use a tripartite graph

Besides tensor-based methods, there are methods that create a tripartite graph composed of user nodes, item nodes, and tag nodes, for predicting rating values [46,47]. We could extend these methods so that they can compute predictions by combining several tripartite graphs (e.g. by combining a user–item–tag graph with an item–class graph and tag–class graph). The tensors can, however, more naturally handle users' activities that are usually represented as multi-object relationships. We can plot the multi-object relationships composed of users, items, and tags as well as rating values in the tensor. For example, as depicted in Fig. 1, we can plot the relationship wherein user u_2 assigns tag “Breathtaking” to item v_2 with a certain rating value as well as the relationship that user u_3 assigns tag “Breathtaking” to item v_3 with another rating value independently in the tensor. On the other hand, a model based on tripartite graphs has the following two drawbacks: (1) It cannot handle each relationship independently. We can explain why this is so by using the example in Fig. 1. The model based on the tripartite graph can handle the relationship wherein user node u_2 is connected to item node v_2 that connects to the tag node “Breathtaking”. It, however, also links user node u_2 to node u_3 through item node v_2 ; this model mixes different types of relationships (user–item–tag relationships and user–item–user relationships) in a graph and thus cannot handle users' activities in the graph so well. (2) It cannot handle the ratings explicitly. Instead, it assigns the weights to edges (edges between user nodes and item nodes as well as edges between item nodes and tag nodes) to represent the rating values in the graph. This, however, is unnatural since the rating value should be assigned to each multi-object relationship (i.e. instance of user activity). Thus, a method based on a tripartite graph is not suitable for computing rating predictions of users' activities.

In detail, the major differences between tensor and tripartite graph is that a triplet is captured by a grid in a 3-D matrices (tensor) using tensor representation; while a triplet is represented as three two-way relations in a tripartite graph. Actually, tensor factorization process makes use of three two-way relations in a tripartite graph. It usually unfolds the tensor across a certain dimension, which intuitively matches the three two-way relations. In addition, with the graph embedding for tripartite graphs, the link prediction can be easily obtained by the inner product of objects' latent vectors.

Furthermore, [27] compares the performance of the matrix-factorization-based method with those of other link prediction methods such as feature-based methods [48,49], graph regularization methods [50,51], and latent class methods [52] when predicting links in the target graph (e.g. user–item graph) by using graphs from other sources (i.e. side information from other sources such as user–user social graph). They concluded that the factorization methods predict links more accurately by solving the imbalance problem in merging heterogeneous graphs (though we pointed out above that the present factorization methods still suffer from the imbalance problem). This paper thus focuses on applying our ideas to the factorization method to improve prediction results for multi-object relationships.

3. Preliminaries

Vocabularies and taxonomies Vocabularies and taxonomies, sometimes called “simple ontologies” [14], are collections of human-defined classes and usually have a hierarchical structure as a graph (vocabulary) or a tree (taxonomy). They are becoming available on the Web as a result of their use in search engines and social networking sites. In particular, the LOD approach aims to link common data structures across the Web. It publishes open data sets using the Resource Description Framework (RDF)⁸ and sets links between data entries from different data sources. As a result, it enables one to search and utilize those RDF links across data sources. This paper uses the vocabularies of DBpedia and Freebase and the taxonomy of WordNet to improve the quality of tensor factorization. DBpedia and Freebase have detailed information about content items such as music, movies, food, and many others. As an example, the music genre “Electronic dance music” in FreeBase is identified by a unique resource identifier (URI)⁹ and is available in RDF format. By referring to this URI, the computer can

⁸ <http://www.w3.org/RDF>.

⁹ http://rdf.freebase.com/rdf/en/electronic_dance_music.

Table 1
Definition of main symbols.

Symbols	Definitions
$\mathcal{R}$	Tensor that includes ratings by users to items with tags.
$\hat{\mathcal{R}}$	The predictive distribution for unobserved ratings by users to items with tags.
α	Observation precision for $\mathcal{R}$.
$\mathbf{u}_m$	m -th user feature vector.
$\mathbf{v}_n$	n -th item feature vector.
$\mathbf{t}_k$	k -th tag feature vector.
$\mathbf{v}'_n$	n -th item feature vector updated by semantic bias.
$\mathbf{t}'_k$	k -th tag feature vector updated by semantic bias.
$\mathbf{U}$	Matrix representation of $\mathbf{u}_m$.
$\mathbf{V}$	Matrix representation of $\mathbf{v}_n$.
$\mathbf{T}$	Matrix representation of $\mathbf{t}_k$.
$\mathbf{V}'$	Matrix representation of $\mathbf{v}'_n$.
$\mathbf{T}'$	Matrix representation of $\mathbf{t}'_k$.
X	Number of different kinds of sparse item (or tag) sets.
$\mathbb{V}_s^{(i)}$	Set of the i -th most sparse items.
$\mathbb{T}_s^{(i)}$	Set of the i -th most sparse tags.
$\mathcal{R}^{v(i)}$	i -th augmented tensor that includes classes of items in $\mathbb{V}_s^{(i)}$.
$\mathcal{R}^{t(i)}$	i -th augmented tensor that includes classes of tags in $\mathbb{T}_s^{(i)}$.
$\mathbf{c}_j^{v(i)}$	j -th semantically biased item feature vector from $\mathcal{R}^{v(i)}$.
$\mathbf{c}_j^{t(i)}$	j -th semantically biased tag feature vector from $\mathcal{R}^{t(i)}$.
$\mathbf{C}^{v(i)}$	Matrix representation of $\mathbf{c}_j^{v(i)}$.
$\mathbf{C}^{t(i)}$	Matrix representation of $\mathbf{c}_j^{t(i)}$.
$S^{v(i)}$	Number of classes that include sparse items in $\mathbb{V}_s^{(i)}$.
$S^{t(i)}$	Number of classes that include sparse tags in $\mathbb{T}_s^{(i)}$.
$f(o)$	Function that returns the classes of object o .
$\delta^{(i)}$	Parameter used for adjusting the number of the i -th most sparsely observed objects.
$\epsilon^{(i)}$	Parameter used for updating the feature vectors for the i -th most sparsely observed items or tags.

acquire the information that “electronic_dance_music” has “electronic_music” as its parent_genre and “pizzicato_five” as one of its artists, and furthermore, it has the owl:sameAs relationship with the URI for “Electronic_Dance_Music” in DBpedia.¹⁰ Objects like the above artists in DBpedia/Freebase can have multiple classes. Because of their graph-based structures, they are defined to be vocabularies.

WordNet is a lexical database for the English language and is often used to support automatic analysis of texts such as user-assigned tags and reviews. In WordNet, each word is classified into one or more synsets, each of which represents a concept and contains a set of words. Thus, we can consider synset as a class and words classified in that synset as instances. An instance can be included in a class. Thus, WordNet’s tree structure defines it to be a taxonomy.

Bayesian probabilistic tensor factorization This paper deals with the relationships formed by user u_m , item v_n , and tag t_k . A third-order tensor $\mathcal{R}$ (definition of main symbols is described in Table 1) is used to model the relationships among objects from sets of users, items, and tags. Here, the (m, n, k) -th element $r_{m,n,k}$ indicates the m -th user’s rating of the n -th item with the k -th tag. This paper assumes that there is at most one observation for each (m, n, k) -th element. This assumption is natural for many applications (i.e. it is rare that a user assigns multiple ratings to the same item using the same tag. Even if the user rates the same item using the same tag several times, we can use the latest rating for such relationships). Tensor factorization assigns a D -dimensional latent feature vector to each user, item, and tag, denoted as $\mathbf{u}_m$, $\mathbf{v}_n$, and $\mathbf{t}_k$, respectively. Here, $\mathbf{u}_m$ is an M -length, $\mathbf{v}_n$ is an N -length, and $\mathbf{t}_k$ is a K -length column vector. Accordingly, each element $r_{m,n,k}$ in $\mathcal{R}$ can be approximated as the inner-product of the three vectors as follows:

$$r_{m,n,k} \approx \langle \mathbf{u}_m, \mathbf{v}_n, \mathbf{t}_k \rangle \equiv \sum_{d=1}^D u_{m,d} \cdot v_{n,d} \cdot t_{k,d}$$

where the index d represents the d -th element of each vector.

BPTF [3] provides a Bayesian treatment for Probabilistic Tensor Factorization (PTF) as the same way the Bayesian Probabilistic Matrix Factorization (BPMF) [53] provides a Bayesian treatment for Probabilistic Matrix Factorization (PMF) [54]. Actually, in [3], the authors provide to use the specific type of third-order tensor (e.g. user–item–time tensor) that has an additional factor for time added to the matrix (e.g. user–item matrix). We, however, consider that BPTF in this paper is an instance of the CANDECOMP/PARAFAC (CP) decomposition [55] and a simple extension of BPMF by adding one more object type (e.g. tag) to the pairs (e.g. user–item matrix) handled by BPMF. BPTF models tensor factorization over a gen-

¹⁰ http://dbpedia.org/resource/Electronic_Dance_Music.

erative probabilistic model for ratings with Gaussian/Wishart priors [56] over parameters. The Wishart distribution is most commonly used as the conjugate prior for the precision matrix in a Gaussian distribution.

We denote the matrix representations of $\mathbf{u}_m$, $\mathbf{v}_n$, and $\mathbf{t}_k$ as $\mathbf{U} \equiv [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_M]$, $\mathbf{V} \equiv [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N]$, and $\mathbf{T} \equiv [\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_K]$. To account for randomness in ratings, BPTF uses a probabilistic model for generating ratings:

$$\mathcal{R}|\mathbf{U}, \mathbf{V}, \mathbf{T} \sim \prod_{m=1}^M \prod_{n=1}^N \prod_{k=1}^K \mathcal{N}(\langle \mathbf{u}_m, \mathbf{v}_n, \mathbf{t}_k \rangle, \alpha^{-1})$$

This equation represents the conditional distribution of $\mathcal{R}$ given $\mathbf{U}$, $\mathbf{V}$, and $\mathbf{T}$ in terms of Gaussian distributions, each having means $\langle \mathbf{u}_m, \mathbf{v}_n, \mathbf{t}_k \rangle$ and precision α .

The generative process of BPTF requires parameters μ_0 , β_0 , $\mathbf{W}_0$, ν_0 , $\tilde{W}_0$, $\tilde{\Lambda}$, and $\tilde{\nu}_0$ in the hyper-priors that should reflect prior knowledge about a specific problem and are treated as constants during training. The process is as follows:

1. Generate $\Lambda_{\mathbf{U}}$, $\Lambda_{\mathbf{V}}$, and $\Lambda_{\mathbf{T}} \sim \mathcal{W}(\Lambda|\mathbf{W}_0, \nu_0)$, where $\Lambda_{\mathbf{U}}$, $\Lambda_{\mathbf{V}}$, and $\Lambda_{\mathbf{T}}$ are the precision matrices (a precision matrix is the inverse of a covariance matrix) for Gaussians. $\mathcal{W}(\Lambda|\mathbf{W}_0, \nu_0)$ is the Wishart distribution of a $D \times D$ random matrix Λ with ν_0 degrees of freedom and a $D \times D$ scale matrix $\mathbf{W}_0$: $\mathcal{W}(\Lambda|\mathbf{W}_0, \nu_0) = \frac{|\Lambda|^{(\nu_0-D-1)/2}}{C} \exp(-\frac{\text{Tr}(\mathbf{W}_0^{-1}\Lambda)}{2})$, where C is a constant.
2. Generate $\mu_{\mathbf{U}} \sim \mathcal{N}(\mu_0, (\beta_0 \Lambda_{\mathbf{U}})^{-1})$, where $\mu_{\mathbf{U}}$ is used as the mean vector for a Gaussian. In the same way, generate $\mu_{\mathbf{V}} \sim \mathcal{N}(\mu_0, (\beta_0 \Lambda_{\mathbf{V}})^{-1})$ and $\mu_{\mathbf{T}} \sim \mathcal{N}(\mu_0, (\beta_0 \Lambda_{\mathbf{T}})^{-1})$, where $\mu_{\mathbf{V}}$ and $\mu_{\mathbf{T}}$ are used as the mean vectors for Gaussians.
3. Generate $\alpha \sim \mathcal{W}(\tilde{\Lambda}|\tilde{W}_0, \tilde{\nu}_0)$.
4. For each $m \in (1 \dots M)$, generate $\mathbf{u}_m \sim \mathcal{N}(\mu_{\mathbf{U}}, \Lambda_{\mathbf{U}}^{-1})$.
5. For each $n \in (1 \dots N)$, generate $\mathbf{v}_n \sim \mathcal{N}(\mu_{\mathbf{V}}, \Lambda_{\mathbf{V}}^{-1})$.
6. For each $k \in (1 \dots K)$, generate $\mathbf{t}_k \sim \mathcal{N}(\mu_{\mathbf{T}}, \Lambda_{\mathbf{T}}^{-1})$.
7. For each non-missing entry (m, n, k) , generate $r_{m,n,k} \sim \mathcal{N}(\langle \mathbf{u}_m, \mathbf{v}_n, \mathbf{t}_k \rangle, \alpha^{-1})$.

Parameters μ_0 , β_0 , $\mathbf{W}_0$, ν_0 , $\tilde{W}_0$, $\tilde{\Lambda}$, and $\tilde{\nu}_0$ should be set properly according to the objective dataset; varying their values, however, has little impact on the final prediction [3].

BPTF views the hyper-parameters α , $\Theta_{\mathbf{U}} \equiv \{\mu_{\mathbf{U}}, \Lambda_{\mathbf{U}}\}$, $\Theta_{\mathbf{V}} \equiv \{\mu_{\mathbf{V}}, \Lambda_{\mathbf{V}}\}$, and $\Theta_{\mathbf{T}} \equiv \{\mu_{\mathbf{T}}, \Lambda_{\mathbf{T}}\}$ as random variables, yielding a predictive distribution for unobserved ratings $\hat{\mathcal{R}}$, which, given an observable tensor $\mathcal{R}$, is:

$$p(\hat{\mathcal{R}}|\mathcal{R}) = \int p(\hat{\mathcal{R}}|\mathbf{U}, \mathbf{V}, \mathbf{T}, \alpha) p(\mathbf{U}, \mathbf{V}, \mathbf{T}, \alpha, \Theta_{\mathbf{U}}, \Theta_{\mathbf{V}}, \Theta_{\mathbf{T}}|\mathcal{R}) d\{\mathbf{U}, \mathbf{V}, \mathbf{T}, \alpha, \Theta_{\mathbf{U}}, \Theta_{\mathbf{V}}, \Theta_{\mathbf{T}}\}. \quad (1)$$

BPTF computes the expectation of $p(\hat{\mathcal{R}}|\mathbf{U}, \mathbf{V}, \mathbf{T}, \alpha)$ over the posterior distribution $p(\mathbf{U}, \mathbf{V}, \mathbf{T}, \alpha, \Theta_{\mathbf{U}}, \Theta_{\mathbf{V}}, \Theta_{\mathbf{T}}|\mathcal{R})$; it approximates the expectation by averaging samples drawn from the posterior distribution. Since the posterior is too complex to directly sample from, it applies the Markov Chain Monte Carlo (MCMC) indirect sampling technique of to infer the predictive distribution for unobserved ratings $\hat{\mathcal{R}}$ (see [3] for the details on the inference algorithm of BPTF).

The time and space complexity of BPTF is $O(\#nz \times D^2 + (M + N + K) \times D^3)$ and is lower than that of typical tensor methods (i.e. GCTF requires $O(M \times N \times K \times D)$ [9]). $\#nz$ is the number of observation entries, and M , N , or K is much greater than D . BPTF can also compute feature vectors in parallel while avoiding fine parameter tuning during the factorization. To initialize the sampling, it creates two-object relationships composed of users and items and takes the maximum a posteriori (MAP) results from probabilistic matrix factorization (PMF) [54]; this lets MCMC converge quickly.

4. Method

Here, we briefly describe the two main ideas behind SSTF and develop these in more detail in successive sections.

4.1. Overview of SSTF

Semantic grounding SSTF resolves ambiguities by linking objects (i.e. items and tags) to semantic classes before conducting tensor factorization. For items (Section 4.2.1), it resolves ambiguities by computing the similarities between the metadata for the items and the properties given to instances in FreeBase/DBpedia. Then, it links items (e.g. v_1 in Fig. 1) to classes (e.g. CV1). For tags (Section 4.2.2), it first classifies tags into those expressing the content of items or users' subjective evaluations about items since they reflect their interests in items. Next, it computes the similarities of the content (subjective) tags and WordNet instances to link tags (e.g. "Breathless") to classes (e.g. CT1). SSTF is also applied to relationships among users, items, and user-assigned reviews (Section 4.2.3). In this case, we replace user-assigned reviews with aspect or subjective phrases from the reviews, treat those phrases as tags, and input the replaced relationships to the semantic grounding part of SSTF. This is because aspect and subjective tags reflect users' opinions of items. SSTF links those tags to DBpedia and WordNet classes for disambiguation. As a result, it outputs a tensor whose objects are linked to semantic classes.

Tensor factorization with semantic augmentation SSTF applies biases from semantic classes to item (or tag) feature vectors in tensor factorization to solve the sparsity problem. First, it creates augmented tensors by incorporating relationships composed of classes of sparse objects (e.g. multi-object relationships composed of “users”, “classes of sparse items”, and “tags” or relationships composed of “users”, “items”, and “classes of sparse tags”) into the original tensor (Section 4.3.1). As an illustration of how this is done, suppose that items v_1 and v_2 in Fig. 1 are sparse items. SSTF creates an augmented tensor by incorporating the relationships composed of u_1 (or u_2), CV1 (or CV2), and “Breathless” into the original tensor. Furthermore, it determines multiple sets of sparse objects according to their degree of sparsity and creates multiple augmented tensors; this gives semantic biases to sparse objects according to the degree of sparsity. Next, it factorizes the original tensor and augmented tensors simultaneously to compute feature vectors for objects and classes (Section 4.3.2). SSTF incorporates semantic biases into the tensor factorization by updating the feature vectors for objects using those for classes. For example, it creates feature vectors for classes, CV1 and CV2, which share semantic knowledge on sparse items, v_1 and v_2 . It then uses the feature vectors for classes, CV1 and CV2, as semantic biases and incorporates them into feature vectors for sparse items, v_1 and v_2 . This procedure aims to solve the sparsity problem.

4.2. Semantic grounding

We now explain semantic grounding by disambiguation.

4.2.1. Linking items to the classes

Content providers often use specific vocabularies to manage items. Each item in the provider usually has several properties (e.g. items in the MovieLens dataset have properties such as the movie’s name and year of release; items in the Last.fm dataset have properties such as the artist’s name and published album name). The vocabularies at the providers are often quite simple; thus, they degrade the tensor factorization with semantic augmentation (explained in next section) after the disambiguation process because such simple vocabularies express only rough-grained semantics for items. Thus, our solution is to link items to instances in the Freebase or the DBPedia vocabularies and resolve the ambiguous items. For example, in our evaluation, the MovieLens original item vocabulary has only 19 classes, while that of Freebase has 360 classes. These additional terms are critical to the success of our method.

We now explain the procedure of the disambiguation: (i) SSTF creates a property vector for item v_n , $\mathbf{p}_{v_n}$, whose elements are the values of the corresponding properties. It also creates a property vector for instance e_j in Freebase or DBPedia, $\mathbf{p}_{e_j}$, which has the same properties as $\mathbf{p}_{v_n}$; (ii) it computes the cosine similarity of the property vectors and identifies the instance that has the highest similarity to v_n . For item disambiguation, this simple strategy works quite well, and SSTF can use the Freebase or DBPedia vocabulary (i.e. genre vocabulary) to classify the item; it links the item object to the values (e.g. Downtempo) of the instances’ property (i.e. Genre).

For example, suppose that music item v_n has the properties “name” and “published album name”; $\mathbf{p}_{v_n}$ is {“AIR”, {“Moon Safari”, “Talkie Walkie”, ..., “The Virgin Suicides”}}. There are several music instances whose names are “AIR” in Freebase; however, there are only individual instances e_j among the published albums named “Moon Safari”, “Talkie Walkie”, and “The Virgin Suicides”. Thus, our method computes that e_j has the highest similarity to v_n , takes the genre property values “Electronica”, “Downtempo”, “Space rock”, “Psychedelic pop”, and “Progressive rock”, and assigns them as v_n ’s classes.

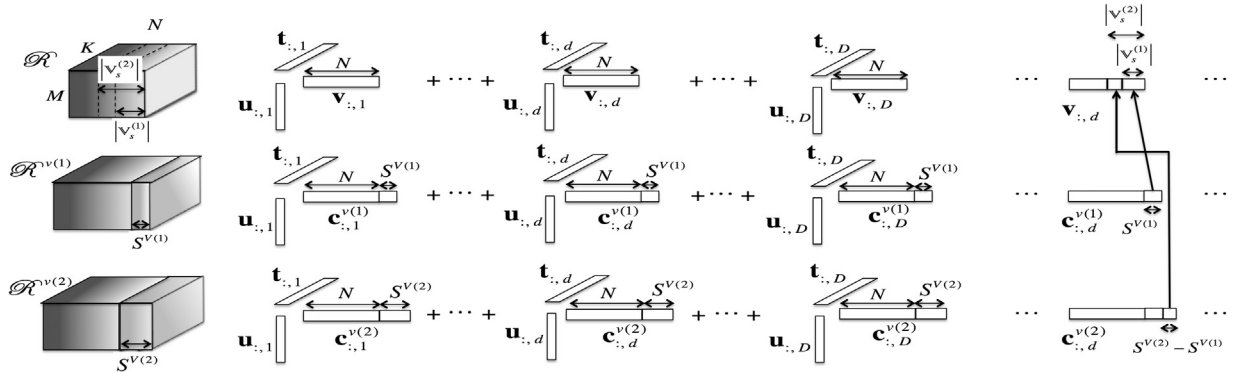
4.2.2. Linking tags to the classes

SSTF classifies tags into those representing the content of items or those indicating the subjectivity of users regarding the items, because [1,46] indicates that such tags are useful in improving prediction accuracy. When determining content tags, noun phrases are extracted from tags because the content tags are usually nouns [46]. Here, SSTF removes stop-words (e.g., conjunctions), transforms the remaining phrases into Parts of Speech (PoS) tuples, and compares the resulting set with the following set of POS-tuple patterns defined for classifying content tags: [<noun>], [<adjective><noun>], and [<determiner><noun>]. In a similar way, we can compare the resulting set with the following set of POS-tuple patterns defined for classifying subjective tags [46]: [<adjective>], [<adjective><noun>], [<adverb>], [<adverb><adjective>], and [<*pronoun>*<adjective>*]. For example, “Bloody dracula” matches the [<adjective><noun>] pattern. SSTF also uses the Stanford-parser [57] to examine negative forms of patterns and identify tags like “Not good”.

We link those extracted tags to vocabulary classes. Because tags are assigned against items, they often reflect the item’s characteristics. Thus, we analyze the properties assigned to the items when linking tags to the classes. Our method takes the following two approaches:

(1) it compares tag t_k and names of Freebase entries indicated by properties of item entry v_n assigned by t_k . If one of the entry names includes t_k , it links t_k to that entry and considers this entry as t_k ’s class. For example, tag “Hanks is great” assigned to movie item “Terminal”, and “Terminal” is related to the “Tom Hanks” entry through the “hasActor” property in Freebase. In this case, it links this tag to the “Tom Hanks” entry.

(2) As for tags that cannot be linked to LOD entries, our method classifies those into WordNet classes. As explained in Section 3, each word in WordNet is classified into one or more concepts. Thus, we should select the most suitable concept for t_k for disambiguation. We use a semantic similarity measurement method [58] based on the vector space model (VSM) method [59] for disambiguation. A tag t_k assigned to item v_n often reflects the item’s characteristics. Accordingly, we use the properties assigned to the items for tag disambiguation. This works as follows: (i) SSTF first crawls the descriptions of



(i) Augmenting tensors. (ii) Simultaneously factorizing tensors into feature vectors while sharing $\mathbf{u}$ and $\mathbf{t}$. (iii) Updating feature vectors.

Fig. 2. Examples of adding semantic biases to item feature vectors when $X=2$. (i) Augmenting tensors (elements in tensors highlighted with lighter color have sparser items). (ii) Factorizing tensors into D -dimensional feature vectors. (iii) Updating feature vectors by using semantically biased feature vectors.

WordNet instances associated with word w in tag t_k . Each WordNet instance w_j has a description d_j . Here, w is a noun if t_k is a content tag and is an adjective or adverb if t_k is a subjective tag. (ii) It next removes some stop-words from the crawled description d_j and constructs a vector $\mathbf{w}_j$ whose elements are words in d_j and whose values are the observed counts of the corresponding words. (iii) It also crawls the item description of v_n and descriptions of v_n 's genres in Freebase/DBpedia. It constructs a vector $\mathbf{i}_n$ whose elements are words in those descriptions and whose values are the observed counts of the corresponding words. Thus, $\mathbf{i}_n$ represents the characteristics of v_n as well as those of v_n 's genres. (iv) After that, it computes the cosine similarity of $\mathbf{i}_n$ and $\mathbf{w}_j$, and finally, it links t_k to the WordNet class that has the instance with the highest similarity value.

SSTF also uses non-content (subjective) tags, though it does not apply semantics to their feature vectors.

4.2.3. Linking reviews to the classes

We use aspect and subjective phrases in the reviews as tags because they reflect the users' opinions of the items [39,60]. Methods of extracting aspect and subjective phrases are described in [39,60]. Of particular note, we use the aspect tags extracted in a semantics-based mining study [39]. This study analyzed reviews in specific objective domains (i.e. music and foods) and extracted aspect tags from review texts that match the instances (i.e. artists and foods) in the vocabulary for that domain. Thus, in the current study, the extracted tags were already linked to the vocabulary. As for subjective tags, we need to disambiguate them. For example, we should analyze how the tag "sweet" is to be understood with regard to "Apple sauce" in the sentence "The apple sauce tasted sweet." SSTF works as follows: (i) it extracts sentences that include aspect tag a_a and subjective tag s_s . It constructs a vector $\mathbf{s}_{a,s}$ whose elements are words in sentences and whose values are the observed counts of the corresponding words. (ii) It crawls the descriptions of WordNet instances associated with word w in s_s . Because s_s is a subjective tag, w is an adjective or adverb. (iii) It constructs a vector $\mathbf{w}_j$ for the crawled description d_j of the WordNet instance w_j and computes the similarity of $\mathbf{s}_{a,s}$ and $\mathbf{w}_j$ in the same way as with tag disambiguation. (iv) It links s_s to the WordNet class that has the WordNet instance with the highest similarity.

4.3. Tensor factorization with semantic augmentation

4.3.1. Semantic augmentation

SSTF lifts sparsely observed objects to their classes and utilizes those classes to augment relationships in a tensor. This allows us to analyze the shared knowledge that has lifted into those classes during the tensor factorization and thereby solve the sparsity problem (see Fig. 2-(i)).

First, we define a set of sparse items, denoted as $\mathbb{V}_s^{(i)}$, for constructing the i -th augmented tensor $\mathcal{R}^{v(i)}$. The set is defined as the group of i -th most sparsely observed items, $v_{s,i}$, among all items. Here, $1 \leq i \leq X$. X is the number of kinds of sets. By investigating X sets in the semantic augmentation process, SSTF can apply a semantic bias to each item feature vector according to its degree of sparsity. We set a 0/1 flag to indicate the existence of relationships composed of user u_m , item v_n , and tag t_k as $o_{m,n,k}$; $\mathbb{V}_s^{(i)}$ is computed as follows:

(1) SSTF first sorts the items from the rarest to the most common and creates a list of items: $\{v_{s(1)}, v_{s(2)}, \dots, v_{s(n-1)}, v_{s(n)}\}$. For example, $v_{s(2)}$ is not less sparsely observed than $v_{s(1)}$.

(2) It iterates the following step (3) from $j=1$ to $j=N$.

(3) If it satisfies the following equation, SSTF adds the j -th sparse item $v_{s(j)}$ to set $\mathbb{V}_s^{(i)}$: $(|\mathbb{V}_s^{(i)}| / \sum_{m,n,k} o_{m,n,k}) < \delta^{(i)}$ where $\mathbb{V}_s^{(i)}$ initially does not have any items and $|\mathbb{V}_s^{(i)}|$ is the number of items in set $\mathbb{V}_s^{(i)}$. If not, it stops the iterations and returns the set $\mathbb{V}_s^{(i)}$ as the group of the i -th most sparsely observed items.

In the above procedure, $\delta^{(i)}$ is a parameter used to determine the number of sparse items in $\mathbb{V}_s^{(i)}$. We also assume that if i is smaller, δ^i will also be smaller, e.g., $\delta^{(1)} < \delta^{(2)}$. A smaller δ^i is used to generate sparser items. Typically, we set $\delta^{(i)}$ to range from 0.05 to 0.20 in accordance with the long-tail characteristic [61] that sparse items account for 5–20% of all observations.

Second, SSTF constructs the i -th augmented tensor $\mathcal{R}^{v(i)}$ by inserting entries for relationships composed of users, classes of sparse items in $\mathbb{V}_s^{(i)}$, and tags into the original tensor $\mathcal{R}$ as follows:

$$r^{v(i)}_{m,j,k} = \begin{cases} r_{m,n,k}, & (1 \leq j \leq N) \cap (j = n) \\ r_{m,s,k}, & (N < j \leq (N + S^{v(i)})) \cap (s_{(j-N)}^v \in f(v_s)) \end{cases}$$

where $f(v_s)$ is a function that returns the classes of item v_s , $S^{v(i)} = |\bigcup_{\mathbb{V}_s^{(i)}} f(v_s)|$ is the number of classes that have the i -th most sparse items, and $s_{(j-N)}^v$ is a class of sparse item v_s where $(j - N)$ represents the identifier of the class ($1 \leq (j - N) \leq S^{v(i)}$). The first line in the above equation indicates that SSTF uses the multi-object relationships composed of user u_m , item v_n , and tag t_k , observed in the original tensor, in creating the augmented tensor if j does not exceed N and j equals n . The second line indicates that SSTF also uses the multi-object relationships composed of user u_m , class $s_{(j-N)}^v$ of sparse item v_s , and tag t_k if j is greater than N and $s_{(j-N)}^v$ is a class of sparse item v_s . There are typically very few observations that have the same user u_m and tag t_k as well as sparse items v_s s that belong to the same class $s_{(j-N)}^v$. Thus, in such cases, we randomly set the rating value for an observation composed of user u_m , class $s_{(j-N)}^v$, and tag t_k among those for observations composed of u_m , v_s , and t_k .

In Fig. 2-(i), SSTF lifts items in the most sparse item set $\mathbb{V}_s^{(1)}$ to their classes (the size is $S^{v(1)}$) and creates an augmented tensor $\mathcal{R}^{v(1)}$ from the original tensor $\mathcal{R}$. It also lifts items in the second-most sparse item set $\mathbb{V}_s^{(2)}$ to their classes (the size is $S^{v(2)}$) and creates an augmented tensor $\mathcal{R}^{v(2)}$ from $\mathcal{R}$.

The set of sparse tags $\mathbb{T}_s^{(i)}$ is defined as the group of most sparsely observed tags among all tags and is computed using the same procedure as the set of sparse items $\mathbb{V}_s^{(i)}$. So we omit the explanation of the procedure for creating $\mathbb{T}_s^{(i)}$. We denote the number of classes that have the i -th most sparse tags as $S^{t(i)} = |\bigcup_{\mathbb{T}_s^{(i)}} f(t_s)|$ and the class of sparse tags as $s_s^{t(i)}$. SSTF constructs the i -th augmented tensor $\mathcal{R}^{t(i)}$ in the same way as it creates $\mathcal{R}^{v(i)}$:

$$r^{t(i)}_{m,n,j} = \begin{cases} r_{m,n,k}, & (1 \leq j \leq K) \cap (j = k) \\ r_{m,n,s}, & (K < j \leq (K + S^{t(i)})) \cap (s_{(j-K)}^t \in f(t_s)) \end{cases}$$

4.3.2. Factorization incorporating semantic biases

SSTF incorporates semantic biases into tensor factorization in the BPTF framework.

Factorization approach We will explain the ideas underlying our factorization method with the help of Fig. 2. For ease of understanding, this figure shows tensors factorized into D -dimensional row vectors, which are contained in matrices $\mathbf{U}$, $\mathbf{V}$, and $\mathbf{T}$ and denoted as $\mathbf{u}_{:,d}$, $\mathbf{v}_{:,d}$, and $\mathbf{t}_{:,d}$ respectively, where $1 \leq d \leq D$. The ideas are as follows:

- (1) SSTF simultaneously factorizes tensors $\mathcal{R}$, $\mathcal{R}^{v(i)}$ ($1 \leq i \leq X$), and $\mathcal{R}^{t(i)}$ ($1 \leq i \leq X$). In particular, it creates feature vectors $\mathbf{c}_j^{v(i)}$ and $\mathbf{c}_j^{t(i)}$ by factorizing $\mathcal{R}^{v(i)}$ and $\mathcal{R}^{t(i)}$ and feature vectors $\mathbf{v}_n$ and $\mathbf{t}_k$ by factorizing $\mathcal{R}$. This enables the semantic biases to be shared during the factorization. In the example shown in Fig. 2-(ii), $\mathcal{R}$, $\mathcal{R}^{v(1)}$, and $\mathcal{R}^{v(2)}$, are simultaneously factorized into D -dimensional feature vectors.
- (2) SSTF uses the same precision α , feature vectors $\mathbf{u}_m$, $\mathbf{v}_n$, and $\mathbf{t}_k$, and their hyper-parameters when factorizing the original tensor and augmented tensors. Thus, the factorization of the original tensor is influenced by those of the augmented tensors through these shared parameters. It lets the factorization of the original tensor be biased by the semantic knowledge in the augmented tensors. In Fig. 2-(ii), $\mathbf{u}_m$ and $\mathbf{t}_k$ are shared among the factorizations of $\mathcal{R}$, $\mathcal{R}^{v(1)}$, and $\mathcal{R}^{v(2)}$.
- (3) SSTF updates the feature vector for item v_n , $\mathbf{v}_n$, to an updated one, $\mathbf{v}'_n$, by incorporating the set of semantically biased feature vectors for v_n 's classes, $\{\mathbf{c}_{(N+j)}^{v(i)}\}_j$, as semantic biases into $\mathbf{v}_n$. This is done by iterating the following equation from $i = 1$ to $i = X$:

$$\mathbf{v}'_n = \begin{cases} (1 - \epsilon^{(i)}) \frac{\sum_{j \in f(v_n)} \mathbf{c}_{(N+j)}^{v(i)}}{|f(v_n)|} & (v_n \in \mathbb{V}_s^{(i)}) \\ \epsilon^{(i)} \mathbf{v}_n & (\text{otherwise}) \end{cases} \quad (2)$$

Here, $\epsilon^{(i)}$ ($0 \leq \epsilon^{(i)} \leq 1$) is a parameter that determines how strongly the semantic biases are to be applied to $\mathbf{v}_n$. Thus, equation (2) returns a vector $\mathbf{v}'_n$ that combines feature vector $\mathbf{v}_n$ and the set of semantically biased feature vectors for v_n 's classes, $\{\mathbf{c}_{(N+j)}^{v(i)}\}_j$, according to the degree of sparsity of item v_n if v_n is a sparse item. Otherwise (if item v_n is a

non-sparse item), it returns $\mathbf{v}_n$. $\mathbf{t}_k$ is updated in a similar way by incorporating semantic biases into the feature vectors of the sparse tags.

In Fig. 2-(iii), each row vector $\mathbf{c}_{:,d}^{v(i)}$ has latent features for N items and those for $S^{v(i)}$ classes. The features for the classes share semantic knowledge on sparse items and so are useful for solving the sparsity problem. SSTF incorporates these features into item-feature vectors according to the degree of sparsity; the features for the most sparse items in $\mathbb{V}_s^{(1)}$ are updated by using $S^{v(1)}$ features. The features for the second-most sparse items in $\mathbb{V}_s^{(2)} - \mathbb{V}_s^{(1)}$ are updated by using $S^{v(2)} - S^{v(1)}$ features.

In this way, SSTF solves the sparsity problem by sharing parameters and updating feature vectors with semantic biases while factorizing tensors.

Generative process The generative process is as follows:

1. Generate $\Lambda_{\mathbf{U}}$, $\Lambda_{\mathbf{V}}$, $\Lambda_{\mathbf{T}}$, $\Lambda_{\mathbf{C}^{v(i)}}$, and $\Lambda_{\mathbf{C}^{t(i)}} \sim \mathcal{W}(\Lambda|\mathbf{W}_0, v_0)$, where $\Lambda_{\mathbf{U}}$, $\Lambda_{\mathbf{V}}$, $\Lambda_{\mathbf{T}}$, $\Lambda_{\mathbf{C}^{v(i)}}$, and $\Lambda_{\mathbf{C}^{t(i)}}$ are the precision matrices for Gaussians. $\mathcal{W}(\Lambda|\mathbf{W}_0, v_0)$ is the Wishart distribution of a $D \times D$ random matrix Λ with v_0 degrees of freedom and a $D \times D$ scale matrix $\mathbf{W}_0$.
2. Generate $\mu_{\mathbf{U}} \sim \mathcal{N}(\mu_0, (\beta_0 \Lambda_{\mathbf{U}})^{-1})$ where $\mu_{\mathbf{U}}$ is the mean vector for a Gaussian. In the same way, generate $\mu_{\mathbf{V}} \sim \mathcal{N}(\mu_0, (\beta_0 \Lambda_{\mathbf{V}})^{-1})$, $\mu_{\mathbf{T}} \sim \mathcal{N}(\mu_0, (\beta_0 \Lambda_{\mathbf{T}})^{-1})$, $\mu_{\mathbf{C}^{v(i)}} \sim \mathcal{N}(\mu_0, (\beta_0 \Lambda_{\mathbf{C}^{v(i)}})^{-1})$, and $\mu_{\mathbf{C}^{t(i)}} \sim \mathcal{N}(\mu_0, (\beta_0 \Lambda_{\mathbf{C}^{t(i)}})^{-1})$, where $\mu_{\mathbf{V}}$, $\mu_{\mathbf{T}}$, $\mu_{\mathbf{C}^{v(i)}}$, and $\mu_{\mathbf{C}^{t(i)}}$ are the mean vectors for Gaussians.
3. Generate $\alpha \sim \mathcal{W}(\tilde{\Lambda}|\tilde{\mathbf{W}}_0, \tilde{v}_0)$.
4. For each $m \in (1 \dots M)$, generate $\mathbf{u}_m$ from $\mathbf{u}_m \sim \mathcal{N}(\mu_{\mathbf{U}}, \Lambda_{\mathbf{U}}^{-1})$.
5. For each $n \in (1 \dots N)$, generate $\mathbf{v}_n$ from $\mathbf{v}_n \sim \mathcal{N}(\mu_{\mathbf{V}}, \Lambda_{\mathbf{V}}^{-1})$.
6. For each $k \in (1 \dots K)$, generate $\mathbf{t}_k$ from $\mathbf{t}_k \sim \mathcal{N}(\mu_{\mathbf{T}}, \Lambda_{\mathbf{T}}^{-1})$.
7. For each $i \in (1, \dots, X)$ and $j \in (1, \dots, (N + S^{v(i)}))$, generate $\mathbf{c}_j^{v(i)}$ from $\mathbf{c}_j^{v(i)} \sim \mathcal{N}(\mu_{\mathbf{C}^{v(i)}}, \Lambda_{\mathbf{C}^{v(i)}}^{-1})$.
8. For each $i \in (1, \dots, X)$ and $j \in (1, \dots, (K + S^{t(i)}))$, generate $\mathbf{c}_j^{t(i)}$ from $\mathbf{c}_j^{t(i)} \sim \mathcal{N}(\mu_{\mathbf{C}^{t(i)}}, \Lambda_{\mathbf{C}^{t(i)}}^{-1})$.
9. For each $n \in (1 \dots N)$, generate an updated feature vector for item v_n , $\mathbf{v}'_n$, by combining $\mathbf{v}_n$ with semantically biased feature vectors for v_n 's classes, $\mathbf{c}_{(N+j)}^{v(i)}$ s, by using Eq. (2).
10. For each $k \in (1 \dots K)$, generate an updated feature vector for tag t_k , $\mathbf{t}'_k$, by combining $\mathbf{t}_k$ with a set of semantically biased feature vectors for t_k 's classes, $\{\mathbf{c}_{(K+j)}^{t(i)}\}_j$, as follows ($(0 \leq \epsilon^{(i)} \leq 1)$ and $(1 \leq i \leq X)$):

$$\mathbf{t}'_k = \begin{cases} (1 - \epsilon^{(i)}) \frac{\sum_{j \in f(t_k)} \mathbf{c}_{(K+j)}^{t(i)}}{|f(t_k)|} & (t_k \in \mathbb{T}_s^{(i)}) \\ \epsilon^{(i)} \mathbf{t}_k & (\text{otherwise}) \end{cases} \quad (3)$$

This equation indicates that if t_k is a sparse tag, $\mathbf{t}'_k$ includes semantic biases from a set of semantically biased feature vectors for t_k 's classes, $\{\mathbf{c}_{(K+j)}^{t(i)}\}_j$, according to t_k 's degree of sparsity. Otherwise, $\mathbf{t}'_k$ does not include semantic biases.

11. For each non-missing entry (m, n, k) , generate $r_{m,n,k}$ as follows:

$$r_{m,n,k} \sim \left(\mathcal{N} \left(\sum_{d=1}^D u_{m,d} \cdot v'_{n,d} \cdot t'_{k,d}, \alpha^{-1} \right) \right)$$

Inferring with Markov Chain Monte Carlo Here, we explain how to compute the predictive distribution for unobserved ratings. Differently from the BPTF model (see Eq. (1)), SSTF should consider the augmented tensors and the degree of sparsity observed for items and tags in order to compute the predictive distribution. Thus, the predictive distribution is computed as follows:

$$p(\hat{\mathcal{R}}|\mathcal{R}, \mathcal{R}^v, \mathcal{R}^t, \mathbf{z}^v, \mathbf{z}^t) = \int p(\hat{\mathcal{R}}|\mathbf{U}, \mathbf{V}, \mathbf{T}, \mathbf{C}^v, \mathbf{C}^t, \mathbf{z}^v, \mathbf{z}^t, \alpha) \\ p(\mathbf{U}, \mathbf{V}, \mathbf{T}, \mathbf{C}^v, \mathbf{C}^t, \Theta_{\mathbf{U}}, \Theta_{\mathbf{V}}, \Theta_{\mathbf{C}^v}, \Theta_{\mathbf{C}^t}, \mathbf{z}^v, \mathbf{z}^t, \alpha | \mathcal{R}, \mathcal{R}^v, \mathcal{R}^t, \mathbf{z}^v, \mathbf{z}^t) \\ d\{\mathbf{U}, \mathbf{V}, \mathbf{T}, \mathbf{C}^v, \mathbf{C}^t, \Theta_{\mathbf{U}}, \Theta_{\mathbf{V}}, \Theta_{\mathbf{T}}, \Theta_{\mathbf{C}^v}, \Theta_{\mathbf{C}^t}, \alpha\}, \quad (4)$$

where $\mathcal{R}^v \equiv \{\mathcal{R}^{v(i)}\}_{i=1}^X$, $\mathcal{R}^t \equiv \{\mathcal{R}^{t(i)}\}_{i=1}^X$, $\mathbf{z}^v \equiv \{\mathbf{z}^{v(i)}\}_{i=1}^X$, $\mathbf{z}^t \equiv \{\mathbf{z}^{t(i)}\}_{i=1}^X$, $\mathbf{C}^v \equiv \{\mathbf{C}^{v(i)}\}_{i=1}^X$, $\mathbf{C}^t \equiv \{\mathbf{C}^{t(i)}\}_{i=1}^X$, $\Theta_{\mathbf{C}^v} \equiv \{\Theta_{\mathbf{C}^{v(i)}}\}_{i=1}^X$, and $\Theta_{\mathbf{C}^t} \equiv \{\Theta_{\mathbf{C}^{t(i)}}\}_{i=1}^X$.

Eq. (4) involves a multi-dimensional integral that cannot be computed analytically. Thus, SSTF views Eq. (4) as the expectation of $p(\hat{\mathcal{R}}|\mathcal{R}, \mathcal{R}^v, \mathcal{R}^t, \mathbf{z}^v, \mathbf{z}^t)$ over the posterior distribution $p(\mathbf{U}, \mathbf{V}, \mathbf{T}, \mathbf{C}^v, \mathbf{C}^t, \Theta_{\mathbf{U}}, \Theta_{\mathbf{V}}, \Theta_{\mathbf{C}^v}, \Theta_{\mathbf{C}^t}, \mathbf{z}^v, \mathbf{z}^t, \alpha | \mathcal{R}, \mathcal{R}^v, \mathcal{R}^t, \mathbf{z}^v, \mathbf{z}^t)$, and it approximates the expectation by using MCMC with the Gibbs sampling paradigm.

MCMC collects a number of samples, L , to approximate the integral in Eq. (4) as follows:

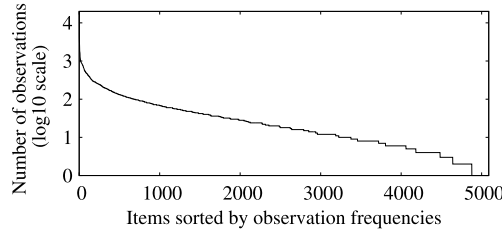


Fig. 3. Log10-scale distribution of item frequencies.

$$p(\hat{\mathcal{R}}|\mathcal{R}, \mathcal{R}^v, \mathcal{R}^t, \mathbf{z}^v, \mathbf{z}^t) \approx \sum_{l=1}^L p(\hat{\mathcal{R}}|\mathbf{U}[l], \mathbf{V}[l], \mathbf{T}[l], \mathbf{C}^v[l], \mathbf{C}^t[l], \mathbf{z}^v[l], \mathbf{z}^t[l], \alpha[l]) \quad (5)$$

where l represents the l -th sample.

It initializes the elements of $\mathbf{U}[1]$, $\mathbf{V}[1]$, $\mathbf{T}[1]$, $\mathbf{C}^{v(i)}[1]$, and $\mathbf{C}^{t(i)}[1]$ by drawing them from a Gaussian distribution. $\mathbf{C}^{v(i)}$ and $\mathbf{C}^{t(i)}$ are matrix representations of $\mathbf{c}_j^{v(i)}$ and $\mathbf{c}_j^{t(i)}$, respectively, and $\mathbf{U}[1]$, $\mathbf{V}[1]$, $\mathbf{T}[1]$, $\mathbf{C}^{v(i)}[1]$, and $\mathbf{C}^{t(i)}[1]$ are the initial states of the matrices $\mathbf{U}$, $\mathbf{V}$, $\mathbf{T}$, $\mathbf{C}^{v(i)}$, and $\mathbf{C}^{t(i)}$, respectively.

Below are the steps in each iteration of the MCMC procedure (Appendix A describes the procedure in more detail):

- (1) Initialize $\mathbf{U}[1]$ and $\mathbf{V}[1]$ by using the MAP results of PMF as per BPTF. Initialize $\mathbf{T}[1]$, $\mathbf{C}^{v(i)}[1]$, and $\mathbf{C}^{t(i)}[1]$ to a Gaussian distribution. Here, $1 \leq i \leq X$.
Next, repeat steps (2) to (6) L times.
- (2) Sample the hyper-parameters in the same way as is done in BPTF, i.e.:
 - $\alpha[l] \sim p(\alpha[l]|\mathbf{U}[l], \mathbf{V}[l], \mathbf{T}[l], \mathcal{R})$
 - $\Theta_{\mathbf{U}}[l] \sim p(\Theta_{\mathbf{U}}[l]|\mathbf{U}[l])$
 - $\Theta_{\mathbf{V}}[l] \sim p(\Theta_{\mathbf{V}}[l]|\mathbf{V}[l])$
 - $\Theta_{\mathbf{T}}[l] \sim p(\Theta_{\mathbf{T}}[l]|\mathbf{T}[l])$
 - $\Theta_{\mathbf{C}^{v(i)}}[l] \sim p(\Theta_{\mathbf{C}^{v(i)}}[l]|\mathbf{C}^{v(i)}[l])$
 - $\Theta_{\mathbf{C}^{t(i)}}[l] \sim p(\Theta_{\mathbf{C}^{t(i)}}[l]|\mathbf{C}^{t(i)}[l])$
 Here, $\Theta_{\mathbf{X}}$ represents $\{\mu_{\mathbf{X}}, \Delta_{\mathbf{X}}\}$. $\mu_{\mathbf{X}}$ and $\Delta_{\mathbf{X}}$ are computed in the same way as in BPTF (see the Preliminaries section).
- (3) Sample the feature vectors in the same way as in BPTF:
 - $\mathbf{u}_m[l+1] \sim p(\mathbf{u}_m[l+1]|\mathbf{V}[l], \mathbf{T}[l], \alpha[l], \Theta_{\mathbf{U}}[l], \mathcal{R})$
 - $\mathbf{v}_n[l+1] \sim p(\mathbf{v}_n[l+1]|\mathbf{U}[l+1], \mathbf{T}[l], \alpha[l], \Theta_{\mathbf{V}}[l], \mathcal{R})$
 - $\mathbf{t}_k[l+1] \sim p(\mathbf{t}_k[l+1]|\mathbf{U}[l+1], \mathbf{V}[l+1], \alpha[l], \Theta_{\mathbf{T}}[l], \mathcal{R})$
- (4) Sample the semantically biased feature vectors by using $\alpha[l]$, $\mathbf{U}[l+1]$, $\mathbf{V}[l+1]$, and $\mathbf{T}[l+1]$ as follows:
 - $\mathbf{c}_j^{v(i)}[l+1] \sim p(\mathbf{c}_j^{v(i)}[l+1]|\mathbf{U}[l+1], \mathbf{T}[l+1], \alpha[l], \Theta_{\mathbf{C}^{v(i)}}[l], \mathcal{R}^v)$
 - $\mathbf{c}_j^{t(i)}[l+1] \sim p(\mathbf{c}_j^{t(i)}[l+1]|\mathbf{U}[l+1], \mathbf{V}[l+1], \alpha[l], \Theta_{\mathbf{C}^{t(i)}}[l], \mathcal{R}^t)$
 Parameters $\alpha[l]$, $\mathbf{U}[l+1]$, $\mathbf{V}[l+1]$, and $\mathbf{T}[l+1]$ are shared in steps (3) and (4).
- (5) Sample the unobserved ratings $\mathcal{R}[l]$ by applying $\mathbf{U}[l+1]$, $\mathbf{V}[l+1]$, $\mathbf{T}[l+1]$, $\mathbf{C}^{v(i)}[l+1]$, $\mathbf{C}^{t(i)}[l+1]$, $\alpha[l]$ to equation (5).
- (6) Calculate $\mathbf{v}'_n[l+1]$ by using Eq. (2) and then set $\mathbf{v}'_n[l+1]$ to $\mathbf{v}_n[l+1]$ in the next iteration. Calculate $\mathbf{t}'_k[l+1]$ by using Eq. (3) and set $\mathbf{t}'_k[l+1]$ to $\mathbf{t}_k[l+1]$ in the next iteration.

In the sixth process, Eqs. (2) and (3) enable us to implement the computation of updates of features of objects ($\mathbf{V}$, $\mathbf{T}$) easily by using the BPTF framework. This is because the features of objects ($\mathbf{V}$, $\mathbf{T}$) can be computed by applying the BPTF to the disambiguated tensor as well as the features of the classes of the objects ($\mathbf{C}^v$ and $\mathbf{C}^t$) can also be computed by applying the BPTF to the augmented tensors. Thus, readers can easily investigate our ideas by using the BPTF's sophisticated implementation. After computing the above features, Eqs. (2) and (3) can merge the features of objects ($\mathbf{V}$, $\mathbf{T}$) and features of their classes ($\mathbf{C}^v$ and $\mathbf{C}^t$) to compute updated features of objects ($\mathbf{V}'$, $\mathbf{T}'$). Besides an easy implementation, this lets us enjoy the fast computation and high scalability of the BPTF framework. This is important since our method uses semantic biases from many features in the classes factorized from multiple kinds of augmented tensors in computing updated features $\mathbf{V}'$ and $\mathbf{T}'$.

4.3.3. Computational complexity and practical issues

The complexity of the original BPTF is $O(\#nz \times D^2 + (M + N + K) \times D^3)$ for each iteration of the MCMC procedure while that of SSTF is $O(\#nz \times 2X \times D^2 + (M + N + K + XN + XK) \times D^3)$. Because the first term is much larger than the rest, the computation time is almost the same as that of BPTF. Furthermore, X is small (less than five). This is because there are many sparse objects and the changes in the observation frequencies among those objects are small (see Figs. 3 and 4, which plot log10-scale distributions of the frequencies of objects in MovieLens.). As a result, SSTF can capture the changes in the

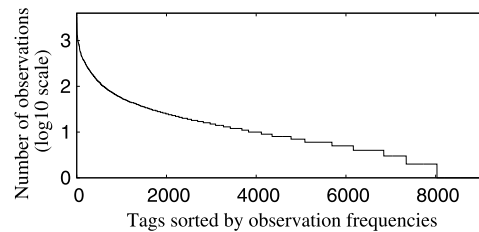


Fig. 4. Log10-scale distribution of tag frequencies.

Table 2
Tag class examples for MovieLens.

C	Breathtaking	Historical	Spirited	Unrealistic
t_k	Breathtaking	Ancient	Racy	Kafkaesque
	Exciting	Historic	Spirited	Surreal
	Thrilling	Past life	Vibrant	Surreal life

observation frequencies of sparse objects by using small X values. Moreover, it can compute the feature vectors in parallel (see Section 4.3.2); their computation is not an onerous burden on modern computers with multi-core CPUs.

The parameter settings are easy. The number of X s is determined by the parameter $\delta^{(i)}$. Accuracy will be much improved simply by setting $\delta^{(1)}$ to 0.05, $\delta^{(2)}$ to 0.10, $\delta^{(3)}$ to 0.15, and $\delta^{(4)}$ to 0.20 to determine sparse sets having long-tail characteristics (see Section 5.4). $\epsilon^{(i)}$ is also easy to set. Here, we initially set $\epsilon^{(i)}$ to 0, where $i \leq X - 1$; only $\epsilon^{(X)}$ needs to be adjusted in order to get very accurate results. This is because it is better to make maximal use of semantic biases in computing predictions for very sparse objects, while it is better to combine the object's feature vector and its semantically biased feature vector when the object is sparse but not very sparse. For example, if X is set to three, SSTF makes maximal use of the semantic biases, which are from the features of classes of objects in the most sparse set $\mathbb{V}_s^{(1)}$ (or $\mathbb{T}_s^{(1)}$) and the second-most sparse set $\mathbb{V}_s^{(2)}$ (or $\mathbb{T}_s^{(2)}$) into features in objects in $(\mathbb{V}_s^{(1)}$ and $\mathbb{V}_s^{(2)}$ (or $\mathbb{T}_s^{(1)}$ and $\mathbb{T}_s^{(2)}$). Thus, $\epsilon^{(1)}$ and $\epsilon^{(2)}$ are set to zero. On the other hand, we adjust $\epsilon^{(3)}$ from 0.1 to 1.0 to combine the object's features and its semantically biased features in detail if the objects are in the third-most sparse set $\mathbb{V}_s^{(3)}$ (or $\mathbb{T}_s^{(3)}$).

5. Evaluation

5.1. Dataset

Our evaluation used the following large-scale datasets:

MovieLens contains user-made ratings of movie items, some of which have user-assigned tags. Ratings range from 0.5 to 5. We extracted a genre vocabulary from Freebase and a tag taxonomy from WordNet. The vocabulary has 360 classes. The taxonomy has 4284 classes. Consequently, it contains 24,565 ratings with 44,595 tag assignments; 33,547 tags have tag classes. The size of the user–item–tag tensor is $2026 \times 5088 \times 9160$. Table 2 shows a taxonomy example.

Yelp contains user-made ratings of restaurants and user reviews of restaurants. Ratings range from 1 to 5. We used the genre vocabulary provided by Yelp as the item vocabulary. It has 179 classes. We extracted aspect/subjective tags from the reviews as follows: (i) food phrases as aspect tags that match the instances in a DBPedia food vocabulary as was done in [39] and (ii) subjective phrases that have relationships with aspect tags (extracted by using the Stanford-parser [60] and the subjective phrases are assumed to be subjective tags if they match the phrases in subjective lexicons [62]). Consequently, we extracted 168,586 subjective and 2,038,560 aspect tags. The aspect and subjective vocabulary contains 4990 distinct tags in 3662 classes. This dataset contains 158,424 ratings with reviews. Among those, 33,863 entries do not contain tags; thus, we assigned dummy tag ids to those entries. In doing so, we could use all reviews in this dataset. The size of the user–item–tag tensor was $36,472 \times 4503 \times 4990$.

Last.fm contains listening counts and tags given by users about artists (items). We computed implicit ratings¹¹ by a user for items by linearly scaling item listening counts such that the maximum value corresponded to 5 and the minimum to 1. This evaluation setting is natural since how strongly users become interested in the artists in the future is very important information for music recommendation services. We extracted a genre vocabulary from Freebase and a tag taxonomy from WordNet. The vocabulary had 38 classes. Each item could have multiple genres. The taxonomy had 2544 classes. We also used items and tags that did not have any classes. As a result, the dataset contained 20,665 ratings and 73,358 tag assignments; 53,964 tags had tag classes. The size of the user–item–tag tensor was $1824 \times 6854 \times 8922$. Table 3 shows an example of the taxonomy.

¹¹ Implicit ratings are often seen in real applications and thus used by evaluations of several rating prediction studies [63,64].

Table 3
Tag class examples for Last.fm.

C	Amazing	Chill	Girl
t_k	Amazing songs	Chill	Girl band
	Awesome cover	Chill zone	Girl groups
	Great singer	Chill wave	Girl power
	Impressive vocals	Ethnic chill	Girl rock

5.2. Compared methods

We compared SSTF with the following methods.

1. *BPMF*, Bayesian probabilistic matrix factorization [53], analyzes ratings by users of items without tags; it cannot predict tags, although the predicted personalized tags can assist users in understanding the predicted items [21].
2. *BPMFI* applies an item vocabulary to *BPMF* as per our approach.
3. *NTF*, Non-negative tensor factorization [65], is a generalization of Non-negative matrix factorization (NMF) [66] and imposes non-negative constraints on tensor and factor matrices. We selected this method since it is a well-known tensor factorization approach and the baseline of the other tensor factorization methods (*VNTF* [67] and *NMTF* [23]) selected for comparison.
4. *BPTF* proposed by [3].
5. *VNTF*, Variational Non-negative Tensor Factorization, is another Bayesian-based tensor factorization method and is a generalization of Variational Non-negative Matrix Factorization (VNMf) [67]. Since we used the *BPTF*, which is the Bayesian-based method, we selected the *VNTF* for comparison.
6. *BPTFT*, which utilizes only tag taxonomy.
7. *BPTFI*, which utilizes only item vocabulary.
8. *SSTFR*, Semantic Sensitive Tensor Factorization with Random disambiguation, links sparse objects (items and tags) to one of their classes randomly (we call this process as random disambiguation). Then it augments the tensors and factorizes the original tensor and augmented tensors simultaneously by using the process explained in Section 4.3.
9. *SRTF* [16] is our previous method that incorporates semantic biases into tensor factorization without sensitively considering the degree of sparsity of objects (here, we set parameter δ used in *SRTF* to 0.2 because it achieved the highest accuracy if we changed δ from 0.0 to 0.2).
10. *GCTF* is the most popular method that factorizes several tensors/matrices simultaneously [8,9].
11. *NMTF* [23] is an another state-of-the-art method that utilizes the auxiliary information like *GCTF*. It factorizes the target and auxiliary tensors simultaneously. The auxiliary data tensors compensate for the sparseness of the target data tensor. Thus, this method is related to our approach and should be compared with *SSTF*.

5.3. Methodology and parameter setup

Following the methodology used in the *BPTF* paper [3], we used root mean square error (RMSE) as the performance metric. It is computed as $\sqrt{(\sum_{i=1}^n (P_i - R_i)^2)/n}$, where n is the number of entries in the test dataset, and P_i and R_i are the predicted and actual ratings of the i -th entry, respectively. Smaller RMSE values indicate higher accuracy. We divided each dataset into three and performed three-fold cross validation. The results below are the average values of the three evaluations. Following [3], the parameters are $\mu_0 = 0$, $v_0 = D$, $\beta_0 = 1$, $\mathbf{W}_0 = \mathbf{I}$, $\tilde{v}_0 = 1$, $\tilde{\Lambda} = 1$, and $\tilde{W}_0 = 1$. L is 500. D was set from 25 to 100. We used the Itakura–Saito (IS) divergence [68] for the optimization function of *GCTF* rather than the IS divergence, Kullback–Leibler (KL) divergence, or Euclidean distance [56] because it achieved the highest accuracy when using ID distance for our datasets. For *VNTF*, we used 200 iterations for learning the target distribution in variational Bayes, since it converges with this number for all datasets. To run *NMTF*, we combined the user–item–tag tensor with item–class and tag–class matrices. We set the weights on the auxiliary matrices used by *NMTF* as 0.01 for MovieLens, 0.1 for Last.fm, and 0.1 for the Yelp dataset since *NMTF* outputs the best RMSE values with the above parameter settings. This weighting parameter determines how strongly the biases from the auxiliary matrices are incorporated in the user–item–tag tensor.

5.4. Results

Sparsity vs. semantic biases First, we investigated the sparseness of the observed objects. Figs. 3 and 4 plot the log10-scale distribution of the item and tag frequencies observed in the MovieLens dataset. From these figures, we can confirm that the item and tag observation frequencies exhibit long-tail characteristics. Thus, the observations are very sparse relative to all possible combinations of observed objects. The distributions of the Yelp dataset had the same tendency.

Next, we investigated the impact of varying X . We changed X from one to four by setting $\delta^{(1)}$ to 0.05, $\delta^{(2)}$ to 0.10, $\delta^{(3)}$ to 0.15, and $\delta^{(4)}$ to 0.20 in determining the sparse set $\mathbb{V}_s^{(i)}$ and $\mathbb{T}_s^{(i)}$. In particular, when X equals one, we set $\delta^{(1)}$ to 0.05.

Table 4

Number of sparse objects in sparse set in each dataset.

i	MovieLens				Yelp				Last.fm			
	1	2	3	4	1	2	3	4	1	2	3	4
Num. of sparse items ($\mathbb{V}_s^{(i)}$)	1913	2327	2663	2915	1980	3052	3389	3833	2119	2691	3065	3326
Num. of sparse tags ($\mathbb{T}_s^{(i)}$)	4271	4271	5295	5295	4257	5172	5643	6255	3463	3880	4070	4186

Table 5RMSE vs. X and $\epsilon^{(X)}$ for MovieLens ($D = 50$).

	$\epsilon^{(X)} = 0.0$	$\epsilon^{(X)} = 0.3$	$\epsilon^{(X)} = 0.6$	$\epsilon^{(X)} = 0.9$
$X = 1$	0.9701	0.9573	0.9491	0.9292
$X = 2$	0.9181	0.9109	0.9012	0.8958
$X = 3$	0.8829	0.8800	0.8781	0.8773
$X = 4$	0.8701	0.8682	0.8646	0.8509

Table 6RMSE vs. X and $\epsilon^{(X)}$ for Yelp ($D = 50$).

	$\epsilon^{(X)} = 0.0$	$\epsilon^{(X)} = 0.3$	$\epsilon^{(X)} = 0.6$	$\epsilon^{(X)} = 0.9$
$X = 1$	1.1770	1.1684	1.1620	1.1432
$X = 2$	1.1447	1.1391	1.1339	1.1293
$X = 3$	1.1265	1.1213	1.1183	1.1169
$X = 4$	1.1150	1.1157	1.1159	1.1135

Table 7RMSE vs. X and $\epsilon^{(X)}$ for Last.fm ($D = 50$).

	$\epsilon^{(X)} = 0.0$	$\epsilon^{(X)} = 0.3$	$\epsilon^{(X)} = 0.6$	$\epsilon^{(X)} = 0.9$
$X = 1$	1.2579	1.2283	1.2192	1.2163
$X = 2$	1.2124	1.2034	1.1983	1.1903
$X = 3$	1.1892	1.1862	1.1732	1.1598
$X = 4$	1.1521	1.1498	1.1387	1.1431

When X equals two, we set $\delta^{(1)}$ to 0.05 and $\delta^{(2)}$ to 0.10. When X equals three, we set $\delta^{(1)}$ to 0.05, $\delta^{(2)}$ to 0.10, and $\delta^{(3)}$ to 0.15. When X equals four, we set $\delta^{(1)}$ to 0.05, $\delta^{(2)}$ to 0.10, $\delta^{(3)}$ to 0.15, and $\delta^{(4)}$ to 0.20.

Table 4 summarizes the average numbers of sparse objects in sparse set $\mathbb{V}_s^{(i)}$ (or $\mathbb{T}_s^{(i)}$). From this table, we can see that there are many sparse objects in the real datasets.

We set $\epsilon^{(i)}$ to 0 except when i was X and only adjusted $\epsilon^{(X)}$, as explained in Section 4.3.3. Tables 5–7 list the RMSE results. The results for MovieLens and Last.fm gradually increased as X and $\epsilon^{(X)}$ increased until they reached a maximum (at $\epsilon^{(X)} = 0.9$ in $X = 4$ for MovieLens and at $\epsilon^{(X)} = 0.6$ in $X = 4$ for Last.fm) and gradually decreased afterwards. These results validate our idea; semantic biases are very useful for dealing with the sparsity problem in tensor factorization. The results for Yelp also showed a gradual improvement as X and $\epsilon^{(X)}$ increased. This tendency can be seen up to a point, $\epsilon^{(X)} = 0.0$ in $X = 4$; however, their accuracy drops after that point. We investigated this effect by setting $\epsilon^{(i)}$ in Eq. (2) and $\epsilon^{(i)}$ in Eq. (3) to different values. As a result, we confirmed that using semantic biases from tag classes gradually improves the accuracy up to the point of $\epsilon^{(X)} = 0.0$ in $X = 4$ and decreases thereafter. On the other hand, semantic biases from item classes improve the results after that point. This indicates that performance can be improved by setting $\epsilon^{(i)}$ for item classes and $\epsilon^{(i)}$ for tag classes independently. In later evaluations, we set $\epsilon^{(X)} = 0.9$ and $X = 4$ for MovieLens, $\epsilon^{(X)} = 0.6$ and $X = 4$ for Last.fm, and $\epsilon^{(X)} = 0.9$ and $X = 4$ for Yelp (for *SSTF*).

Accuracy Next, we compared the accuracies of the methods. The RMSE values are shown in Tables 8–10. We will start by describing the results comparing *BPTF* with *BPMF*. Note that we did not evaluate *BPMF* and *BPMFI* on the Last.fm dataset because the metrics for the listening frequencies for the user–item–tag (triple) relationships and those for the user–item (double) relationships are completely different. On the other hand, the ratings were assigned manually by users to those relationships, so the *BPMF* results may help readers to understand our results. *BPTF* is less accurate than *BPMF*. This indicates that *BPTF* does not utilize the additional information (tags/reviews) for prediction so well because the observations in the tensors are much sparser than those in the matrices. Note that the evaluations in the original *BPTF* paper [3] analyzed multi-object relationships composed of users, items, and time stamps. It achieved slightly higher accuracy than *BPMF*. This is because the number of time stamps in their evaluations was only about 30; thus, the tensors evaluated in [3] are much more dense than ours. On the other hand, the tensors we analyze in our evaluation is very sparse; the ratio of observations to all possible elements in the tensor for MovieLens is $4.7 \cdot 10^{-7}$, that for YELP is $1.9 \cdot 10^{-7}$, and that for LastFM is $6.6 \cdot 10^{-7}$. *BPMFI* is also less accurate than *BPMF* on the MovieLens and Yelp datasets because the observations in the user–item

Table 8
RMSE for MovieLens.

	<i>BPMF</i>	<i>BPMFI</i>	<i>NTF</i>	<i>BPTF</i>	<i>VNTF</i>	<i>BPTFI</i>	<i>BPTFT</i>	<i>SSTFR</i>	<i>SRTF</i>	<i>NMTF</i>	<i>SSTF</i>
$D = 25$	0.9072	0.9140	2.1464	0.9976	1.874	0.8972	0.9660	0.8769	0.8787	1.7677	0.8681
$D = 50$	0.9058	0.9118	2.0795	0.9701	1.9838	0.8809	0.9550	0.8658	0.8721	1.7475	0.8509
$D = 75$	0.9071	0.9115	2.068	0.9737	2.13	0.8860	0.9523	0.8687	0.8718	1.7561	0.8582
$D = 100$	0.9045	0.9125	2.0637	0.9649	2.2059	0.8810	0.9461	0.8702	0.8725	1.7686	0.8591

Table 9
RMSE for Yelp.

	<i>BPMF</i>	<i>BPMFI</i>	<i>NTF</i>	<i>BPTF</i>	<i>VNTF</i>	<i>BPTFI</i>	<i>BPTFT</i>	<i>SSTFR</i>	<i>SRTF</i>	<i>NMTF</i>	<i>SSTF</i>
$D = 25$	1.1631	1.1676	1.8853	1.2269	1.4728	1.1523	1.1702	1.1471	1.1479	2.2718	1.1456
$D = 50$	1.1660	1.1692	1.8279	1.1770	1.473	1.1289	1.1524	1.1141	1.1148	1.9773	1.1135
$D = 75$	1.1639	1.1679	1.8131	1.1796	1.4963	1.1272	1.1538	1.1118	1.1121	1.9155	1.1105
$D = 100$	1.1638	1.1677	1.8161	1.1801	1.555	1.1287	1.1572	1.1132	1.1169	1.8915	1.1127

Table 10
RMSE for Last.fm.

	<i>BPMF</i>	<i>BPMFI</i>	<i>NTF</i>	<i>BPTF</i>	<i>VNTF</i>	<i>BPTFI</i>	<i>BPTFT</i>	<i>SSTFR</i>	<i>SRTF</i>	<i>NMTF</i>	<i>SSTF</i>
$D = 25$	–	–	1.8853	1.2579	1.5371	1.1670	1.1738	1.1583	1.1560	1.7240	1.1432
$D = 50$	–	–	1.8279	1.2507	1.7451	1.1523	1.1652	1.1468	1.1482	1.683	1.1387
$D = 75$	–	–	1.8131	1.2652	1.9149	1.1583	1.1632	1.1505	1.1512	1.6726	1.1428
$D = 100$	–	–	1.8161	1.2578	2.235	1.1572	1.1629	1.1482	1.1488	1.6711	1.1412

matrices are not so sparse in these cases. Conversely, *BPTFI*, *BPTFT*, and of course, *SSTF* are much more accurate than *BPTF* on all datasets. That is, semantic biases are especially useful in solving the sparsity problem.

Interestingly, *SSTFR* was more accurate than *BPTF* even though its disambiguation strategy was simple. We investigated the disambiguation results and found the following reasons: (1) WordNet synsets with the same names often share similar meanings. So, if we use a well-defined taxonomy like WordNet, the random disambiguation process and semantic biases after disambiguation improve accuracy; the improvement resulting from semantic biases exceeds the decrease created by mistakes due to random disambiguation. (2) Most item objects are not ambiguous. Moreover, semantic biases derived from item classes improve accuracy more than those from tag classes, as indicated by the results of *BPTFI* and *BPTFT*. To summarize, *SSTFR* can achieve high accuracy if a well-defined tag taxonomy like WordNet is available and there are few ambiguous items, as in our dataset.

SSTF was more accurate than *SSTFR*. To assess those results in detail, we compared the accuracy of our disambiguation algorithm with that of random disambiguation. We focused on the linkage results from the objects (items/tags) that have multiple Freebase or Wordnet candidate instances and randomly picked 100 linkage results from those candidates for MovieLens and those for LastFM. We compared the accuracies of our disambiguation method with those of random disambiguation. The expert judges the correct answers of the linkage results and there is a correct answer for each linkage result in our evaluation. The results of our method are about 72% for MovieLens and 64% for Last.fm, while the results of random disambiguation are 31% for MovieLens and 41% for Last.fm. Thus, we can understand that a higher quality disambiguation can improve the overall prediction accuracy when incorporating semantic information into tensor factorization.

SSTF also outperformed *SRTF*. This means that the results are improved by factorizing multiple augmented tensors and computing semantic biases carefully according to the degree of sparsity of objects.

Next, we investigated the results of *NTF* and found that they are much worse than other methods for all datasets. The pure *NTF* cannot handle sparse tensors like in our evaluation datasets though the real datasets typically contain such sparse relationships. The results of *VNTF* are mostly better than those of *NTF*; however, they are worse than those of *BPTF*. This is mainly because the non-negative constraints in factorizations have a limitation in predicting the rating values as [69] reported in their evaluation; they cannot distinguish positive ratings (represented as non-negative values) from negative ones (represented as negative values). *VNTF* is better than *NTF* since the Bayesian-based factorization approach typically outputs better predictions than do non-Bayesian-based methods [3,53].

The RMSEs of *NMTF* are mostly better than those of *NTF*, but much worse than those of *BPTF* and those of *SSTF*. This is mainly because non-negative constraints of *NMTF* have a limitation in predicting the rating values. Other reasons are: (1) *NMTF* straightforwardly combines different relationships, i.e., rating relationships among users, items, and tags, link relationships among items and their classes, and link relationships among tags and their classes. Thus, it suffers from the balance problem. (2) *NMTF* gives biases derived from auxiliary information to all relationships involving three objects. It cannot deal with the sparsity problem in a satisfactory way. (3) *NMTF* uses the KL divergence for optimizing the predictions since its authors are interested in “discrete value observations such as stars in product reviews”, as described in [23]. Our datasets, especially MovieLens and YELP, are those they are interested in; however, exponential family distributions like Poisson distribution do not fit our rating datasets so well, as explained in the above paragraph. As for the Last.fm dataset,

Table 11
Comparing *BPTF*/*SSTF* ($D = 50$) with *GCTF*.

MovieLens			Yelp			Last.fm		
<i>BPTF</i>	<i>SSTF</i>	<i>GCTF</i>	<i>BPTF</i>	<i>SSTF</i>	<i>GCTF</i>	<i>BPTF</i>	<i>SSTF</i>	<i>GCTF</i>
0.9298	0.8728	0.9952	1.7923	1.1820	1.4227	1.1687	1.1286	1.1892

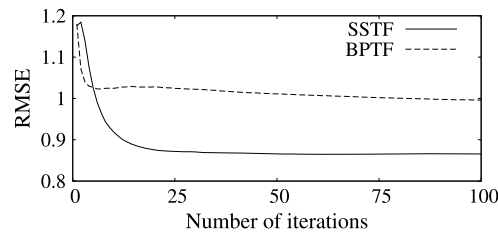


Fig. 5. RMSE vs. number of iterations ($D = 50$).

the implicit ratings used in the dataset are generated from the listening counts and follow a power law distribution. *NMTF*, however, has worse accuracy than *SSTF*. Surprisingly, the RMSEs of *NMTF* are much worse than those of *VNTF* for the YELP dataset. We investigated the non-sparse tags in the Yelp dataset and found that there are a few tags that have too many observations such as rice and bread. *NMTF* shares the biases from such popular tags with other tags via their food categories. The biases from such non-sparse tags, however, become noise in prediction, and thus, the accuracy of *NMTF* for the Yelp dataset was poor.

On the other hand, *SSTF* naturally incorporates semantic classes into tensors after fitting them to user–item–tag tensors. *SSTF* also carefully selects sparse objects and gives semantic biases to them. Moreover, the Gaussian prior used by *SSTF* naturally fits prediction tasks of rating values including implicit ratings in the Last.fm dataset. Thus, *SSTF* is superior to *NMTF* for rating prediction task. We should also note that *NMTF* needs complicated parameters, i.e., the weights on the auxiliary matrices, to effectively utilize the auxiliary information in tensor factorization. This is because it associates objects (items) with tensors and matrices, each of which treats different types of information; the balance problem in handling heterogeneous datasets from different sources arises; thus, setting the parameters of *NMTF* is more difficult than that of *SSTF*.

Next, we compared the accuracies of *BPTF*, *SSTF*, and *GCTF*. Because *GCTF* required much more memory than *BPTF* and *SSTF* (see [9]), we could not apply it to the whole evaluation dataset on our computer. Accordingly, we randomly picked five sets of 200 users in each dataset and evaluated the accuracy for each set. Table 11 shows the average RMSE values of those evaluations. *SSTF* was more accurate than *GCTF*. This is because, for the same reasons as its superiority to *NMTF*, *SSTF* naturally incorporates semantic classes into tensors after fitting them to user–item–tag tensors; *GCTF* straightforwardly combines different types of relationships, rating relationships among users, items, and tags, link relationships among items and their classes, and link relationships among tags and their classes. *SSTF* also carefully selects sparse objects and gives semantic biases to them while *GCTF* gives biases derived from side information to all relationships involving three objects. Different from *NMTF*, *GCTF* can use the KL divergence, IS divergence, and Euclidean distance for optimization model (the Euclidean distance is usually used for rating prediction, as is the Gaussian distribution), however, it is worse than *SSTF* (as described in Section 5.3, *GCTF* achieves the highest accuracy when it uses the IS distance). Thus, the above results indicate that our ideas are superior to those of other state-of-the-art tensor-based methods, regardless of the distributions of priors used by these methods.

The improvement in accuracy had by *SSTF* over the other methods was statistically significant: $\alpha < 0.05$. *SSTF* was up to 12% more accurate than *BPTF* for MovieLens (*SSTF* was 0.8509, while *BPTF* was 0.9649).

We also evaluated the accuracy of MovieLens while varying the number of iterations in the MCMC procedure (see Fig. 5). The results indicated that the accuracy of *SSTF* converged quickly. The results for Yelp showed the same tendency.

To summarize, *SSTF* achieved much higher accuracy than state-of-the-art methods like *GCTF*, *NMTF*, *VNTF* and *BPTF* in evaluations using large-scale datasets in real applications; *SSTF* can thus be considered one of the best tensor methods for the rating prediction task.

Impact of prediction results Table 12 shows examples of the differences between the predictions output by *BPTF* and those by *SSTF* for Yelp. The “Prediction” column presents values predicted by *BPTF* and by *SSTF*, while the “Actual” column presents actual ratings given by users as found in the test dataset.

User “1” highly rated the food tag “udon” at restaurant “A” and rated “falafel” in restaurant “B” as fair. In the test dataset, he highly rated “ramen” at restaurant “C” and “sangaki” in restaurant “D”. The tags “udon” and “ramen” as well as the restaurants “B” and “D” are sparsely observed in the training dataset. *SSTF* accurately predicted those selections because it uses the semantic knowledge that “udon” and “ramen” are both in the tag class “Japanese noodles”; user “1” likes this type of food. Note that restaurant “C” is not a Japanese restaurant. Thus, this selection was accurately predicted with the

Table 12Example prediction for Yelp acquired by *BPTF* and *SSTF* (bold sentences represent food tags).

Training dataset				
User	Tag in review sentence	Item/main genre	Rating	
1	The tempura udon was delicious.	A/Japanese	4.0	
1	The falafel is okay – but pretty hard.	B/Greek	3.0	
Predictions by BPTF and SSTF				
User	Tag in review sentence	Item/main genre	Prediction	Actual
1	The ramen was really good.	C/Hawaiian	5.0/4.1	4.0
1	Flaming saganaki is fabulous.	D/Greek	2.9/3.9	4.0
Training dataset				
User	Tag in review sentence	Item/main genre	Rating	
2	The beef shawarma kebab is my favorite.	E/Mediterranean	5.0	
3	The iced coffee lacked condensed milk . Bad.	F/Thai	2.0	
Predictions by BPTF and SSTF				
User	Tag in review sentence	Item/main genre	Prediction	Actual
2	The tabbouleh is excellent.	G/Middle Eastern	2.6/3.8	4.0
2	The red curry is good. Not awesome.	F/Thai	4.9/3.4	3.0

Table 13Computation time (minutes) when $L = 500$.

	$D = 50$			$D = 100$		
	<i>MovieLens</i>	<i>Yelp</i>	<i>Last.fm</i>	<i>MovieLens</i>	<i>Yelp</i>	<i>Last.fm</i>
<i>BPTF</i>	5.95	22.22	6.70	11.40	42.26	12.80
<i>SSTF</i>	25.43	48.89	30.38	46.09	98.46	63.98

tag class, but not the item class. *SSTF* also uses the semantic knowledge that user “1” often rated restaurants in the “Greek” class in the training dataset and restaurant “D” tended to be highly rated by users who like the “Greek” class.

User “2” highly rated the food tag “shawarma” at restaurant “E”. In the test dataset, he highly rated “tabbouleh” at restaurant “G”. The tags “shawarma” and “tabbouleh” are sparsely observed in the training dataset. *SSTF* predicted this selection accurately because it used the semantic knowledge that they are in the same tag class “Arab cuisine”. User “2” also rated as fair the “red curry” at restaurant “F” in the test dataset. The training dataset had users who rated restaurant “F” fairly or highly with sparse food tags “red curry”, “green curry”, and “yellow curry” in the class “thai curry”. There are also users who did not highly rate “non-curry dishes” at restaurant “F” (see an example review by user “3”). *SSTF* can use all observations for “thai curry” at restaurant “F” while excluding those for “non-curry dishes” in computing the predictions. Thus, it predicted the selection accurately. The predictions of *BPTF* were inaccurate because it could not use any semantics underlying objects.

Our disambiguation process is pre-computed before the tensor factorization, and our augmentation process is quite simple; they can be easily applied to other tensor factorization schemes. For example, we confirmed that our approach works well with *VNTF*. Thus, we believe our ideas can enhance the prediction accuracy for various applications.

Computation time Table 13 presents the computation times of *BPTF* and *SSTF* when $L = 500$. All experiments were conducted on a Linux 3.33 GHz Intel Xeon (24 cores) server with 192 GB of main memory. All methods were implemented with Matlab and GCC. We can see that the computation time of *SSTF* is shorter than the case that increases linearly in proportion to $2X$ (the number of different kinds of sparse item sets and sparse tag sets.). Furthermore, we can set L smaller than 500 (see Fig. 5) and computation time is linear in L . Thus, we can conclude that *SSTF* can compute more accurate predictions quickly; it works better than *BPTF* on real applications.

6. Conclusion

This is the first study to use the semantics underlying objects to enhance tensor factorization accuracy. Semantic sensitive tensor factorization (*SSTF*) is critical to using semantics to analyze human activities in detail, a key AI goal. It creates semantically enhanced tensors by assessing sparsely observed objects and factorizes the tensors simultaneously in the *BPTF* framework. Experiments showed that *SSTF* is up to 12% more accurate than current methods and can support many applications.

We are now applying SSTF to link prediction in social networks (e.g. predicting the frequency of future communications among users). Accordingly, we are applying our idea to VNTE, as described in the evaluation section, because the frequency of communications among users often follows an exponential family distribution. Another interesting direction for future work is predicting user activities among cross-domain applications such as music and movie rating services. We think our ideas have potential for cross-domain analysis because they can use semantic knowledge in the format of LOD shared among several service domains. A third interesting direction is development of methods that handle more detailed semantic knowledge than simple vocabularies/taxonomies. In the current study which follows the previous paper [1], we think the “genre” relationships seemed to be useful for solving the sparsity problem caused by the sparse item set. Thus, we used this type of semantic relationship as a first step to solving the sparsity problem in tensor factorization. However, that there are many other useful relationships between different classes in the ontologies. Actually, some of those were heuristically selected in our conference paper [16] (e.g. values of instance properties such as “directedBy” and “actedBy” in the movie ontology). They improved the prediction accuracy, because they reflected users’ interests in the item objects. We also think that there are many useless relationships (e.g. “<<http://dbpedia.org/ontology/timeZone>>” and “<<http://dbpedia.org/ontology/spouse>>”) that usually do not reflect users’ interests in the item objects. Developing a method that automatically selects useful relationships and incorporates semantics from them would improve the prediction accuracy and be an interesting direction of future work. Furthermore, we are interested in improving the model design that reflects the merge strategy of object features and the features of its classes in Eq. (2) more naturally in the way that samples features updated with semantic biases from the posteriors.

Appendix A. Learning parameters using MCMC

This section explains how to learn feature vectors and their hyperparameters in the MCMC procedure of SSTF. The following notations and item numbers are the same as those used in Section 4.3.2.

- (1) Initialize $\mathbf{U}[1]$, $\mathbf{V}[1]$ by using the MAP results of PMF as per BPTF. Initialize $\mathbf{T}[1]$, $\mathbf{C}^{v(i)}[1]$, and $\mathbf{C}^{t(i)}[1]$ by using a Gaussian distribution. $\mathbf{C}^{v(i)}$ and $\mathbf{C}^{t(i)}$ are matrix representations of $\mathbf{c}_j^{v(i)}$ and $\mathbf{c}_j^{t(i)}$, respectively. Repeat steps (2) to (6) L times and sample feature vectors and their hyperparameters. Each sample in the MCMC procedure depends only on the previous one.
- (2) Sample the conditional distribution of α given $\mathcal{R}$, $\mathbf{U}$, $\mathbf{V}$, and $\mathbf{T}$ (as per BPTF) as follows:

$$\begin{aligned}
 p(\alpha|\mathcal{R}, \mathbf{U}, \mathbf{V}, \mathbf{T}) &= \mathcal{W}(\alpha|\tilde{\mathbf{W}}_0^*, \nu_0^*), \\
 \nu_0^* &= \tilde{\nu}_0 + \sum_{k=1}^K \sum_{n=1}^N \sum_{m=1}^M 1, \\
 (\tilde{\mathbf{W}}_0^*) &= (\tilde{\mathbf{W}}_0)^{-1} + \sum_{k=1}^K \sum_{n=1}^N \sum_{m=1}^M (r_{m,n,k} - \langle \mathbf{u}_m, \mathbf{v}_n, \mathbf{t}_k \rangle)^2
 \end{aligned} \tag{A.1}$$

Sample hyperparameters $\Theta_{\mathbf{U}}$, $\Theta_{\mathbf{V}}$, and $\Theta_{\mathbf{T}}$ as per BPTF. They are all computed in the same way. For example, $\Theta_{\mathbf{V}}$ is conditionally independent of all other parameters given $\mathbf{V}$:

$$\begin{aligned}
 p(\Theta_{\mathbf{V}}|\mathbf{V}) &= \mathcal{N}(\boldsymbol{\mu}_{\mathbf{V}}|\boldsymbol{\mu}_0^*, (\beta_0^* \boldsymbol{\Lambda}_{\mathbf{V}})^{-1}) \mathcal{W}(\boldsymbol{\Lambda}_{\mathbf{V}}|\mathbf{W}_0^*, \nu_0^*), \\
 \boldsymbol{\mu}_0^* &= \frac{\beta_0 \boldsymbol{\mu}_0 + N \bar{\mathbf{v}}}{\beta_0 + N}, \quad \beta_0^* = \beta_0 + N, \quad \nu_0^* = \nu_0 + N, \\
 \mathbf{W}_0^{*-1} &= \mathbf{W}_0^{-1} + N \bar{\mathbf{S}} + \frac{\beta_0 N}{\beta_0 + N} (\nu_0 - \bar{\mathbf{v}})(\nu_0 - \bar{\mathbf{v}})^{tr}, \\
 \bar{\mathbf{v}} &= \frac{1}{N} \sum_{n=1}^N \mathbf{v}_n, \quad \bar{\mathbf{S}} = \frac{1}{N} \sum_{n=1}^N (\mathbf{v}_n - \bar{\mathbf{v}})(\mathbf{v}_n - \bar{\mathbf{v}})^{tr}.
 \end{aligned} \tag{A.2}$$

The MCMC procedure also samples the hyperparameters of $\mathbf{C}^{v(i)}$, $\Theta_{\mathbf{C}^{v(i)}}$, in order to generate ratings for $\mathcal{R}^{v(i)}$. The generative process of $\mathcal{R}^{v(i)}$ is almost the same as that of $\mathcal{R}$ except for sharing α , $\mathbf{u}_m$, and $\mathbf{t}_k$, which are also used in sampling $\mathbf{v}_n$ when sampling $\mathbf{c}_j^{v(i)}$ (see steps (3) and (4)). Thus, $\Theta_{\mathbf{C}^{v(i)}}$ can be computed in the same way as $\Theta_{\mathbf{V}}$ as follows:

$$\begin{aligned}
 p(\Theta_{\mathbf{C}^{v(i)}}|\mathbf{C}^{v(i)}) &= \mathcal{N}(\boldsymbol{\mu}_{\mathbf{C}^{v(i)}}|\boldsymbol{\mu}_0^*, (\beta_0^* \boldsymbol{\Lambda}_{\mathbf{C}^{v(i)}})^{-1}) \mathcal{W}(\boldsymbol{\Lambda}_{\mathbf{C}^{v(i)}}|\mathbf{W}_0^*, \nu_0^*), \\
 \boldsymbol{\mu}_0^* &= \frac{\beta_0 \boldsymbol{\mu}_0 + (N + S^{v(i)}) \bar{\mathbf{c}}^{v(i)}}{\beta_0 + N + S^{v(i)}}, \quad \beta_0^* = \beta_0 + N + S^{v(i)}, \quad \nu_0^* = \nu_0 + N + S^{v(i)},
 \end{aligned}$$

$$\begin{aligned}
\mathbf{W}_0^{*-1} &= \mathbf{W}_0^{-1} + (N + S^{v(i)})\bar{\mathbf{S}} + \frac{\beta_0(N + S^{v(i)})}{\beta_0 + N + S^{v(i)}}(\mathbf{v}_0 - \mathbf{c}^{v(i)})(\mathbf{v}_0 - \mathbf{c}^{v(i)})^T, \\
\mathbf{c}^{v(i)} &= \frac{1}{N + S^{v(i)}} \sum_{j=1}^{N+S^{v(i)}} \mathbf{c}^{v(i)}, \\
\bar{\mathbf{S}} &= \frac{1}{N + S^{v(i)}} \sum_{j=1}^{N+S^{v(i)}} (\mathbf{c}_j^{v(i)} - \mathbf{c}^{v(i)})(\mathbf{c}_j^{v(i)} - \mathbf{c}^{v(i)})^T.
\end{aligned} \tag{A.3}$$

- (3) Sample model parameters $\mathbf{U}$, $\mathbf{V}$, $\mathbf{T}$ as per BPTF. They are all computed in the same way. For example, $\mathbf{V}$ can be factorized into individual items. If $\mathbf{P}_{m,k} \equiv \mathbf{u}_m \cdot \mathbf{t}_k$, each item feature vector is computed in parallel as follows:

$$\begin{aligned}
p(\mathbf{v}_n | \mathcal{R}, \mathbf{U}, \mathbf{T}, \alpha, \Theta_{\mathbf{V}}) &= \mathcal{N}(\mathbf{v}_n | \boldsymbol{\mu}_n^*, (\boldsymbol{\Lambda}_n^*)^{-1}), \\
\boldsymbol{\mu}_n^* &\equiv (\boldsymbol{\Lambda}_n^*)^{-1}(\boldsymbol{\Lambda}_V \boldsymbol{\mu}_V + \alpha \sum_{k=1}^K \sum_{m=1}^M r_{m,n,k} \mathbf{P}_{m,k}), \\
\boldsymbol{\Lambda}_n^* &\equiv \boldsymbol{\Lambda}_V + \alpha \sum_{k=1}^K \sum_{m=1}^M o_{m,n,k} \mathbf{P}_{m,k} \mathbf{P}_{m,k}^T.
\end{aligned} \tag{A.4}$$

- (4) Sample model parameters $\mathbf{c}^{v(i)}$ and $\mathbf{c}^{t(i)}$. The conditional distribution of $\mathbf{c}^{v(i)}$ can be factorized into individual items and individual augmented item classes. Thus, each feature vector for items and item classes can be computed in parallel. The computation is as follows:

$$\begin{aligned}
p(\mathbf{c}_j^{v(i)} | \mathcal{R}^{v(i)}, \mathbf{U}, \mathbf{T}, \alpha, \Theta_{\mathbf{c}^{v(i)}}) &= \mathcal{N}(\mathbf{c}_j^{v(i)} | \boldsymbol{\mu}_j^{(i)*}, (\boldsymbol{\Lambda}_j^{(i)*})^{-1}), \\
\boldsymbol{\mu}_j^{(i)*} &\equiv (\boldsymbol{\Lambda}_j^{(i)*})^{-1}(\boldsymbol{\Lambda}_{\mathbf{c}^{v(i)}} \boldsymbol{\mu}_{\mathbf{c}^{v(i)}} + \alpha \sum_{n=1}^N \sum_{m=1}^M r^{v(i)}_{m,j,k} \mathbf{P}_{m,k}), \\
\boldsymbol{\Lambda}_j^{(i)*} &\equiv \boldsymbol{\Lambda}_{\mathbf{c}^{v(i)}} + \alpha \sum_{k=1}^K \sum_{m=1}^M o_{m,j,k}^{v(i)} \mathbf{P}_{m,k} \mathbf{P}_{m,k}^T.
\end{aligned} \tag{A.5}$$

Here, $o_{m,j,k}^{v(i)}$ is a 0/1 flag to indicate the existence of a relationship composed of user u_m , the j -th item/class that corresponds to $\mathbf{c}_j^{v(i)}$, and a tag t_k in $\mathcal{R}^{v(i)}$.

Steps (3) and (4) share precision α and feature vectors $\mathbf{u}_m$ and $\mathbf{t}_k$ (in $\mathbf{P}_{m,k}$). This means SSTF shares those parameters in the factorization of tensors $\mathcal{R}$ and $\mathcal{R}^{v(i)}$. Thus, semantic biases can be shared among the above tensors via the shared parameters. The conditional distribution of $\mathbf{c}^{t(i)}$ can be computed in the same way.

- (5) In each iteration, samples the unobserved ratings $\hat{\mathcal{R}}[l]$ by applying $\mathbf{U}[l+1]$, $\mathbf{V}[l+1]$, $\mathbf{T}[l+1]$, $\mathbf{C}^{v(i)}[l+1]$, $\mathbf{C}^{t(i)}[l+1]$, $\alpha[l]$ to equation (5).
(6) Calculate $\mathbf{v}_n'[l+1]$ by using Eq. (2) and then set $\mathbf{v}_n'[l+1]$ to $\mathbf{v}_n[l+1]$ in the next iteration. Calculate $\mathbf{t}_k'[l+1]$ by using Eq. (3) and then set $\mathbf{t}_k'[l+1]$ to $\mathbf{t}_k[l+1]$ in the next iteration.

In this way, SSTF effectively incorporates semantic biases into the feature vectors of sparse objects in each iteration of the MCMC procedure.

References

- [1] M. Nakatsuji, Y. Fujiwara, Linked taxonomies to capture users' subjective assessments of items to facilitate accurate collaborative filtering, *Artif. Intell.* 207 (2014) 52–68.
- [2] A. Karatzoglou, X. Amatriain, L. Baltrunas, N. Oliver, Multiverse recommendation: n-dimensional tensor factorization for context-aware collaborative filtering, in: *Proceedings of the 4th ACM Conference on Recommender Systems, RecSys, 2010*, pp. 79–86.
- [3] L. Xiong, X. Chen, T.-K. Huang, J.G. Schneider, J.G. Carbonell, Temporal collaborative filtering with bayesian probabilistic tensor factorization, in: *Proceedings of the SIAM International Conference on Data Mining, SDM, 2010*, pp. 211–222.
- [4] G.A. Miller, Wordnet: a lexical database for English, *Commun. ACM* 38 (1995) 39–41.
- [5] J. Hu, L. Fang, Y. Cao, H.-J. Zeng, H. Li, Q. Yang, Z. Chen, Enhancing text clustering by leveraging Wikipedia semantics, in: *Proceedings of the 31st Annual International ACM SIGIR Conference, SIGIR 2008, 2008*, pp. 179–186.
- [6] E. Gabrilovich, S. Markovitch, Computing semantic relatedness using Wikipedia-based explicit semantic analysis, in: *Proceedings of the 20th International Joint Conference on Artificial Intelligence, IJCAI, 2007*, pp. 1606–1611.
- [7] A. Narita, K. Hayashi, R. Tomioka, H. Kashima, Tensor factorization using auxiliary information, in: *Proceedings of the Machine Learning and Knowledge Discovery in Databases – European Conference, ECML/PKDD 2011, 2011*, pp. 501–516.
- [8] Y.K. Yilmaz, A.-T. Cemgil, U. Simsekli, Generalised coupled tensor factorisation, in: *Proceedings of the Annual Conference on Neural Information Processing Systems, NIPS, 2011*, pp. 2151–2159.

- [9] B. Ermiş, E. Acar, A.T. Cemgil, Link prediction via generalized coupled tensor factorisation, CoRR, arXiv:1208.6231, 2012.
- [10] B. Ermiş, E. Acar, A.T. Cemgil, Link prediction in heterogeneous data via generalized coupled tensor factorization, Data Min. Knowl. Discov. (2013).
- [11] V.W. Zheng, B. Cao, Y. Zheng, X. Xie, Q. Yang, Collaborative filtering meets mobile recommendation: a user-centered approach, in: Proceedings of the 24th AAAI Conference on Artificial Intelligence, AAAI, 2010.
- [12] E. Acar, T.G. Kolda, D.M. Dunlavy, All-at-once optimization for coupled matrix and tensor factorizations, CoRR, arXiv:1105.3422, 2011.
- [13] C. Bizer, T. Heath, T. Berners-Lee, Linked data – the story so far, Int. J. Semantic Web Inf. Syst. 5 (2009) 1–22.
- [14] D.L. McGuinness, Ontologies come of age, in: Spinning the Semantic Web, MIT Press, 2003, pp. 171–194.
- [15] C. Bizer, J. Lehmann, G. Kobilarov, S. Auer, C. Becker, R. Cyganiak, S. Hellmann, Dbpedia – a crystallization point for the web of data, J. Web Semant. 7 (2009) 154–165.
- [16] M. Nakatsuji, Y. Fujiwara, H. Toda, H. Sawada, J. Zheng, J.A. Hendler, Semantic data representation for improving tensor factorization, in: Proceedings of the 28th AAAI Conference on Artificial Intelligence, AAAI, 2014.
- [17] T. Franz, A. Schultz, S. Sizov, S. Staab, Triplerank: ranking semantic web data by tensor decomposition, in: Proceedings of the 8th International Semantic Web Conference, ISWC, 2009, pp. 213–228.
- [18] M. Nickel, V. Tresp, H.-P. Kriegel, Factorizing yago: scalable machine learning for linked data, in: Proceedings of the 21st International World Wide Web Conference, WWW, 2012, pp. 271–280.
- [19] S. Rendle, Z. Gantner, C. Freudenthaler, L. Schmidt-Thieme, Fast context-aware recommendations with factorization machines, in: Proceedings of the 34th Annual International ACM SIGIR Conference, SIGIR, 2011, pp. 635–644.
- [20] S. Rendle, L. Schmidt-Thieme, Pairwise interaction tensor factorization for personalized tag recommendation, in: Proceedings of the 3rd ACM International Conference on Web Search and Data Mining, WSDM, 2010, pp. 81–90.
- [21] S. Rendle, L.B. Marinho, A. Nanopoulos, L. Schmidt-Thieme, Learning optimal ranking with tensor factorization for tag recommendation, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD, 2009, pp. 727–736.
- [22] X. Liu, K. Aberer, Soco: a social network aided context-aware recommender system, in: Proceedings of the 22nd International World Wide Web Conference, WWW, 2013, pp. 781–802.
- [23] K. Takeuchi, R. Tomioka, K. Ishiguro, A. Kimura, H. Sawada, Non-negative multiple tensor factorization, in: Proceedings of the 13th International Conference on Data Mining, ICDM, 2013, pp. 1199–1204.
- [24] D. Rafailidis, A. Nanopoulos, Modeling the dynamics of user preferences in coupled tensor factorization, in: Proceedings of the 8th ACM Conference on Recommender Systems, RecSys, 2014, pp. 321–324.
- [25] Y. Chen, C. Hsu, H.M. Liao, Simultaneous tensor decomposition and completion using factor priors, IEEE Trans. Pattern Anal. Mach. Intell. 36 (2014) 577–591.
- [26] W. Liu, J. Chan, J. Bailey, C. Leckie, K. Ramamohanarao, Mining labelled tensors by discovering both their common and discriminative subspaces, in: Proceedings of the SIAM International Conference on Data Mining, SDM 2013, 2013, pp. 614–622.
- [27] A.K. Menon, C. Elkan, Link prediction via matrix factorization, in: Proceedings of the Machine Learning and Knowledge Discovery in Databases – European Conference Part II, ECML/PKDD 2011 (2), 2011, pp. 437–452.
- [28] U. Simsekli, A.T. Cemgil, Score guided musical source separation using generalized coupled tensor factorization, in: Proceedings of the 20th European Signal Processing Conference, EUSIPCO, 2012, pp. 2639–2643.
- [29] B. Ermiş, A.T. Cemgil, N.B. Marvasti, B. Acar, Liver CT annotation via generalized coupled tensor factorization, in: Working Notes for Conference and Labs of the Evaluation Forum, CLEF, 2014, pp. 421–427.
- [30] E. Acar, G. Gürdeniz, M.A. Rasmussen, D. Rago, L.O. Dragsted, R. Bro, Coupled matrix factorization with sparse factors to identify potential biomarkers in metabolomics, Int. J. Knowl. Discov. Bioinformatics 3 (2012) 22–43.
- [31] Understanding data fusion within the framework of coupled matrix and tensor factorizations, Chemom. Intell. Lab. Syst. 129 (2013) 53–63.
- [32] C. Li, Q. Zhao, J. Li, A. Cichocki, L. Guo, Multi-tensor completion with common structures, in: Proceedings of the 29th AAAI Conference on Artificial Intelligence, AAAI, 2015, pp. 2743–2749.
- [33] W. Chu, Z. Ghahramani, Probabilistic models for incomplete multi-dimensional arrays, J. Mach. Learn. Res. 5 (2009) 89–96.
- [34] A. Beutel, P.P. Talukdar, A. Kumar, C. Faloutsos, E.E. Papalexakis, E.P. Xing, Flexifact: scalable flexible factorization of coupled tensors on hadoop, in: Proceedings of the 2014 SIAM International Conference on Data Mining, SDM, 2014, pp. 109–117.
- [35] U. Simsekli, B. Ermiş, A.T. Cemgil, F. Oztoprak, S.I. Birbil, Parallel and distributed inference in coupled tensor factorization models, in: Proceedings of the Distributed Machine Learning and Matrix Computations Workshop on the Annual Conference on Neural Information Processing Systems, NIPS, 2014.
- [36] E.E. Papalexakis, C. Faloutsos, T.M. Mitchell, P.P. Talukdar, N.D. Sidiropoulos, B. Murphy, Turbo-smt: accelerating coupled sparse matrix-tensor factorizations by 200x, in: Proceedings of the SIAM International Conference on Data Mining, SDM, 2014, pp. 118–126.
- [37] D. Krompass, M. Nickel, V. Tresp, Large-scale factorization of type-constrained multi-relational data, in: Proceedings of the International Conference on Data Science and Advanced Analytics, DSAA 2014, 2014, pp. 18–24.
- [38] M. Nakatsuji, Y. Miyoshi, Y. Otsuka, Innovation detection based on user-interest ontology of blog community, in: Proceedings of the 5th International Semantic Web Conference, ISWC, 2006, pp. 515–528.
- [39] M. Nakatsuji, M. Yoshida, T. Ishida, Detecting innovative topics based on user-interest ontology, J. Web Semant. 7 (2009) 107–120.
- [40] M. Nakatsuji, Y. Fujiwara, A. Tanaka, T. Uchiyama, K. Fujimura, T. Ishida, Classical music for rock fans?: novel recommendations for expanding user interests, in: Proceedings of the 19th ACM International Conference on Information and Knowledge Management, CIKM, 2010, pp. 949–958.
- [41] M. Nakatsuji, Y. Fujiwara, T. Uchiyama, K. Fujimura, User similarity from linked taxonomies: subjective assessments of items, in: Proceedings of the International Joint Conference on Artificial Intelligence, IJCAI, 2011, pp. 2305–2311.
- [42] M. Nakatsuji, Y. Fujiwara, T. Uchiyama, H. Toda, Collaborative filtering by analyzing dynamic user interests modeled by taxonomy, in: Proceedings of the 11th International Semantic Web Conference, ISWC 2012, 2012, pp. 361–377.
- [43] Y. Koren, Collaborative filtering with temporal dynamics, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD, 2009, pp. 447–456.
- [44] A. Nakaya, T. Katayama, M. Itoh, K. Hiranaka, S. Kawashima, Y. Moriya, S. Okuda, M. Tanaka, T. Tokimatsu, Y. Yamanishi, A.C. Yoshizawa, M. Kanehisa, S. Goto, Kegg oc: a large-scale automatic construction of taxonomy-based ortholog clusters, Nucleic Acids Res. (2013) 353–357.
- [45] S.A. Khan, S. Kaski, Bayesian multi-view tensor factorization, in: Proceedings of the Machine Learning and Knowledge Discovery in Databases – European Conference Part I, ECML/PKDD (1), 2014, pp. 656–671.
- [46] I. Cantador, I. Konstas, J.M. Jose, Categorising social tags to improve folksonomy-based recommendations, J. Web Semant. 9 (2011) 1–15.
- [47] I. Konstas, V. Stathopoulos, J.M. Jose, On social networks and collaborative recommendation, in: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR, 2009, pp. 195–202.
- [48] C. Wang, V. Satuluri, S. Parthasarathy, Local probabilistic models for link prediction, in: Proceedings of the 7th IEEE International Conference on Data Mining, ICDM, 2007, pp. 322–331.
- [49] M.A. Hasan, V. Chaoji, S. Salem, M. Zaki, Link prediction using supervised learning, in: Proceedings of the SIAM Data Mining Conference 2006 Workshop on Link Analysis, Counterterrorism and Security, 2006.
- [50] H. Kashima, T. Kato, Y. Yamanishi, M. Sugiyama, K. Tsuda, Link propagation: a fast semi-supervised learning algorithm for link prediction, in: Proceedings of the SIAM International Conference on Data Mining, SDM, 2009, pp. 1100–1111.

- [51] R. Raymond, H. Kashima, Fast and scalable algorithms for semi-supervised link prediction on static and dynamic graphs, in: Machine Learning and Knowledge Discovery in Databases, European Conference Part III, ECML/PKDD (3), 2010, pp. 131–147.
- [52] E.M. Airoldi, D.M. Blei, S.E. Fienberg, E.P. Xing, Mixed membership stochastic blockmodels, *J. Mach. Learn. Res.* 9 (2008) 1981–2014.
- [53] R. Salakhutdinov, A. Mnih, Bayesian probabilistic matrix factorization using Markov chain Monte Carlo, in: Proceedings of the 25th International Conference on Machine Learning, ICML, vol. 25, 2008, pp. 880–887.
- [54] R. Salakhutdinov, A. Mnih, Probabilistic matrix factorization, in: Proceedings of the Annual Conference on Neural Information Processing Systems, NIPS, vol. 20, 2008.
- [55] T.G. Kolda, B.W. Bader, Tensor decompositions and applications, *SIAM Rev.* 51 (2009) 455–500.
- [56] C.M. Bishop, Pattern Recognition and Machine Learning (Information Science and Statistics), Springer-Verlag, New York, 2006.
- [57] D. Klein, C.D. Manning, Accurate unlexicalized parsing, in: Proceedings of the 41st Annual Meeting of the Association for Computational Linguistics, ACL, 2003, pp. 423–430.
- [58] J.G. Zheng, L. Fu, X. Ma, P. Fox, SEM+: discover “same as” links among the entities on the web of data, Technical report, Rensselaer Polytechnic Institute, 2013, <http://tw.rpi.edu/web/doc/Document?uri=http://tw.rpi.edu/media/2013/10/07/e293/SEM.doc>.
- [59] P.D. Turney, P. Pantel, From frequency to meaning: vector space models of semantics, *J. Artif. Intell. Res.* 37 (2010) 141–188.
- [60] J. Yu, Z.-J. Zha, M. Wang, T.-S. Chua, Aspect ranking: identifying important product aspects from online consumer reviews, in: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, HLT, 2011, pp. 1496–1505.
- [61] C. Anderson, The Long Tail: Why the Future of Business Is Selling Less of More, Hyperion, 2006.
- [62] M. Hu, B. Liu, Mining and summarizing customer reviews, in: Proceedings of the 10th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD, 2004, pp. 168–177.
- [63] J. Sun, S. Papadimitriou, C. Yung Lin, N. Cao, S. Liu, W. Qian, Multivis: content-based social network exploration through multi-way visual analysis, in: Proceedings of the SIAM Data Mining Conference, SDM, 2009, pp. 1064–1075.
- [64] Y. Hu, Y. Koren, C. Volinsky, Collaborative filtering for implicit feedback datasets, in: Proceedings of the 8th IEEE International Conference on Data Mining, ICDM, 2008, pp. 263–272.
- [65] A. Shashua, T. Hazan, Non-negative tensor factorization with applications to statistics and computer vision, in: Proceedings of the International Conference on Machine Learning, ICML, 2005, pp. 792–799.
- [66] D.D. Lee, H.S. Seung, Learning the parts of objects by nonnegative matrix factorization, *Nature* 401 (1999) 788–791.
- [67] A.T. Cemgil, Bayesian inference for nonnegative matrix factorisation models, *Comput. Intell. Neurosci.* 2009 (2009) 4:1–4:17.
- [68] F. Itakura, S. Saito, Analysis synthesis telephony based on the maximum likelihood method, in: Proceedings of the 6th International Congress on Acoustics, vol. 17, 1968, pp. C17–C20.
- [69] Z. Wang, Yongji Wang, Hu Wu, Tags meet ratings: improving collaborative filtering with tag-based neighborhood method, in: Proceedings of the Workshop on Social Recommender Systems, SRS 2010, 2010, pp. 15–23.